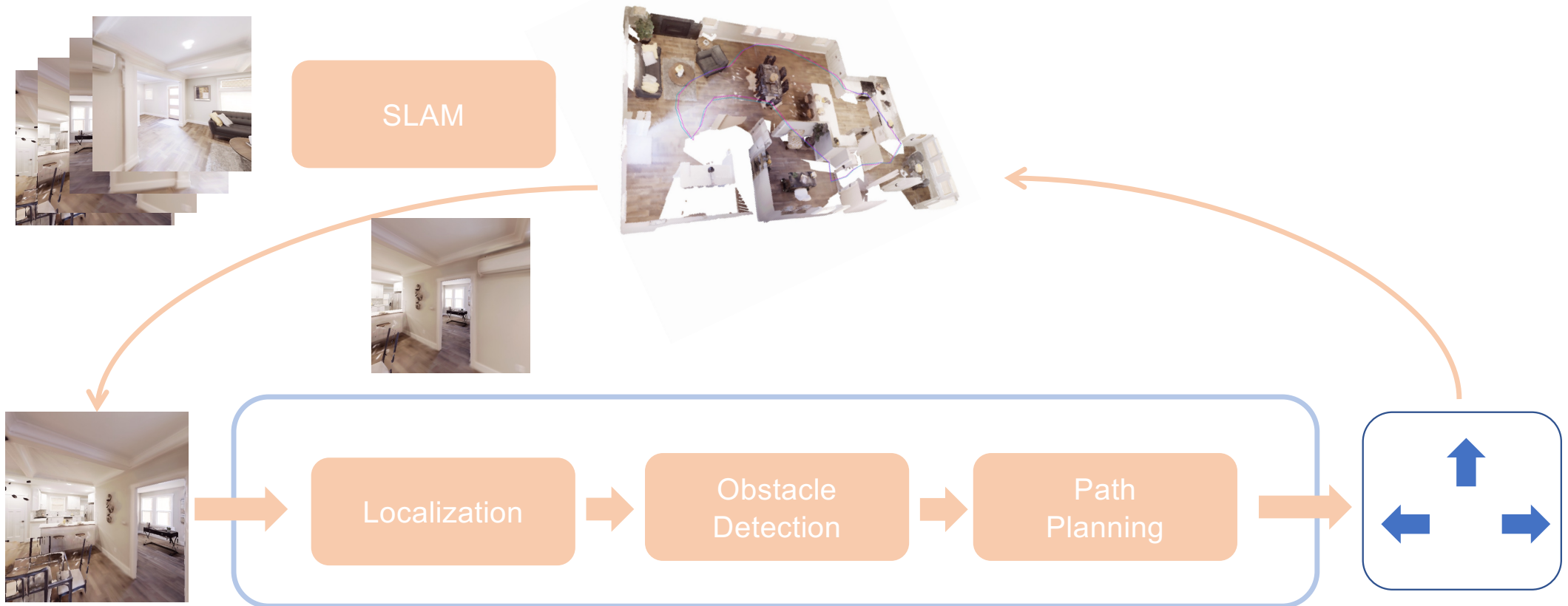
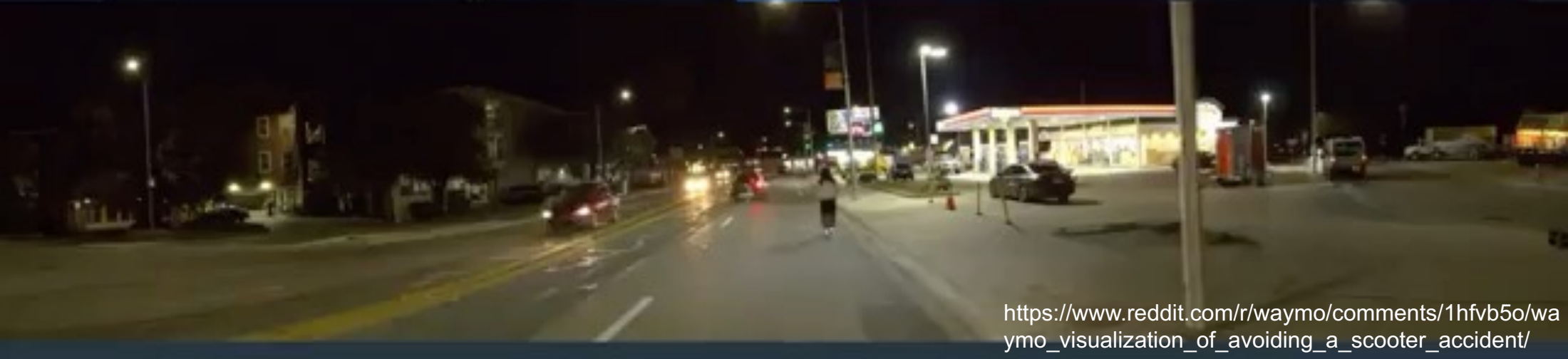


Week	Date	Topic	Note
1	2/23~2/27	Introduction	
2	3/2~3/6	Recent Advances in Physical AI	
3	3/9~3/13	Data Pyramid in Physical AI	HW1 Release
4	3/16~3/20	Introduction to Universal Manipulation Interface (UMI) and LeRobot	
5	3/23~3/27	Simultaneous Localization and Mapping	HW2 Release
6	3/30~4/3	Depth Estimation	4/2: No lecture due to Qingming Festival
7	4/6~4/10	Path Planning	4/6: No lecture due to Qingming Festival HW3 Release
8	4/13~4/17	Robotic Manipulation	4/13: No lecture; the lecture will be pre-recorded.
9	4/20~4/24	Introduction to Robot Operating System (ROS) and ICRA'26 WBCD Challenge	HW4 Release and course project spec release
10	4/27~5/1	Imitation Learning/Reinforcement Learning for Robotics Manipulation	
11	5/4~5/8	Vision-Language Action Models	5/4: No lecture; the lecture will be pre-recorded.
12	5/11~5/15	Introduction to Ontology Engineering	
13	5/18~5/22	Ontology-based Evaluation	HW5 Release
14	5/25~5/29	Scenario-based Safety Validation and Policy Evaluation	
15	6/1~6/5	Research Frontiers	6/1: the lecture will be online; 6/4: No lecture
16	6/8~6/12	*Course Project Presentation*	

Classical Robot Navigation System





https://www.reddit.com/r/waymo/comments/1hfvb5o/waymo_visualization_of_avoiding_a_scooter_accident/

What is Path Planning?

*Dynamics: the relationship between the forces acting on a robot and the resulting motion of the robot

- Path planning is the process of computing a feasible path from an initial state to a target state, while satisfying constraints such as:
 - avoiding obstacles
 - minimizing cost (distance or time)
- A pure geometric problem of finding a collision-free path without concerning the dynamics*, the duration of motion, or constraints on the motion*

Motion Planning concerns dynamics, duration of motion, and constraints on the motion and the control inputs

Problem Formulation

- Given an initial state $q(s) = q_{\text{start}}$ and a desired final state q_{goal} , find a collision-free path $q(s) \in \mathcal{C}_{\text{free}}$, where $s \in [0, 1]$.
- $q(s)$: Path
- $\mathcal{C}_{\text{free}}$: free configuration space

Path vs Trajectory vs Policy

- Path
 - geometric
 - no time
- Trajectory
 - time-parameterized
 - could includes velocity, acceleration
- Policy
 - mapping: state $\rightarrow$ action
 - used in reinforcement learning / learning-based planning

Types of Path Planning

1. Graph-Based Methods

- Discretize space into nodes and edges
- Algorithms:
 - Dijkstra's Algorithm
 - A* search algorithm
- Good for structured environments

2. Sampling-Based Methods

- Randomly explore space instead of full discretization
- Algorithms:
 - Rapidly-exploring Random Tree
 - Probabilistic Roadmap
- Scales well to high-dimensional robotics problems

Types of Path Planning

3. Optimization-Based Methods

- Formulate as minimizing a cost function
- Produces smooth, realistic trajectories
- Representative algorithms:
 - Trajectory optimization
 - Model predictive control (MPC)
 - Potential field

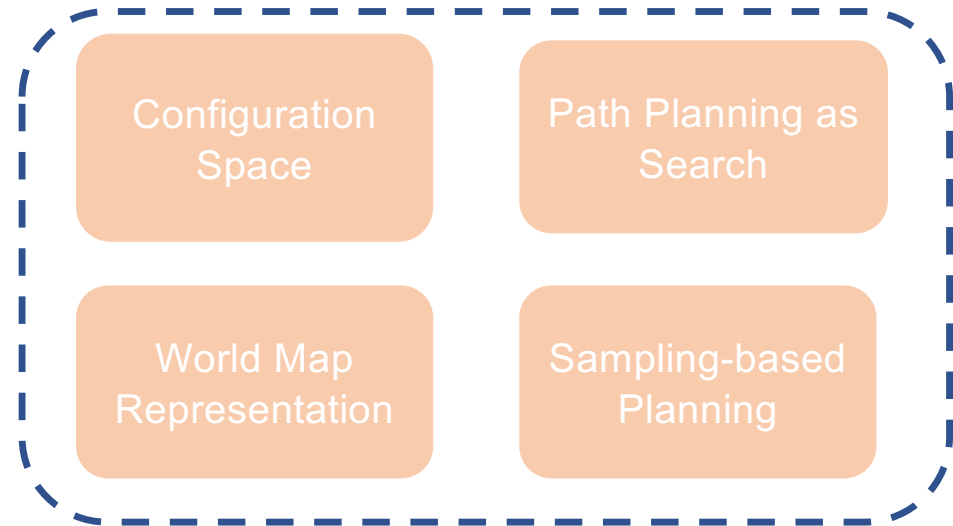
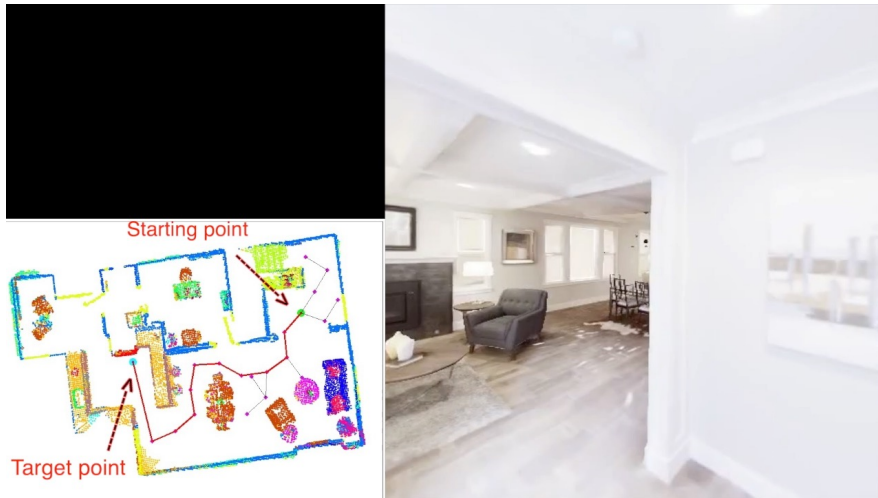
4. Learning-Based Methods

- Use deep learning or reinforcement learning
- Learn policies from data or simulation
- Algorithms:
 - Imitation learning
 - Reinforcement learning
 - Generative models

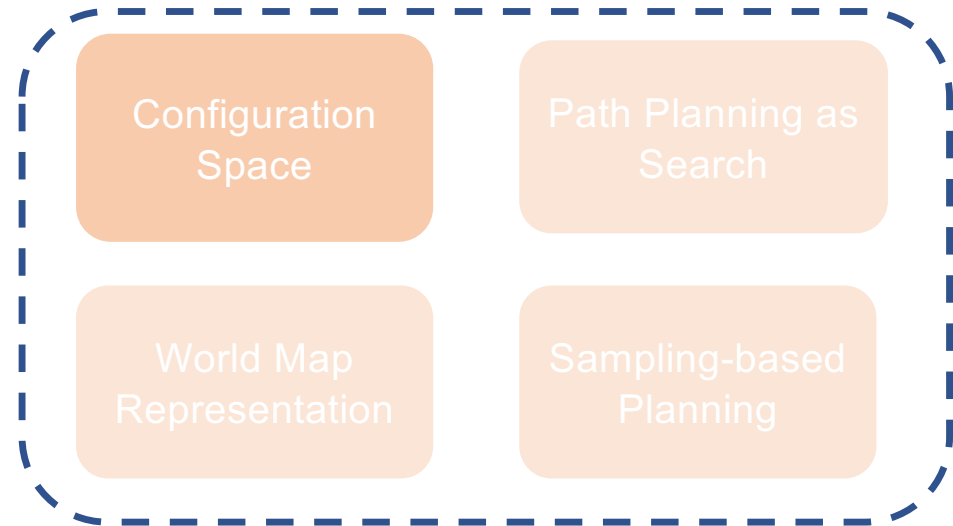
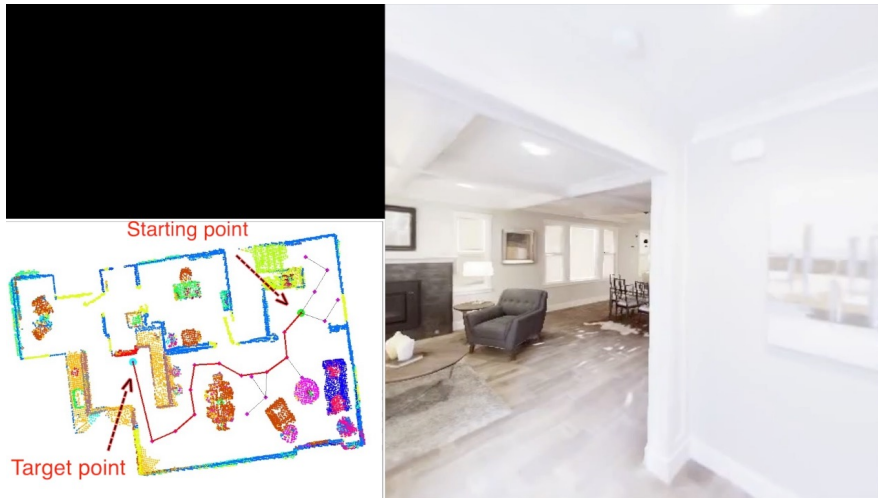
Key Components

- **State Space:** All possible configurations
- **Start & Goal:** Where you are → where you want to go
- **Obstacles:** Things you must avoid
- **Cost Function:** What you want to optimize (e.g., shortest path)
- **Constraints:** Physical limits (e.g., joint limits)

Concept Map for Path Planning



Concept Map for Path Planning



Representation Robot and Environment Configuration Space

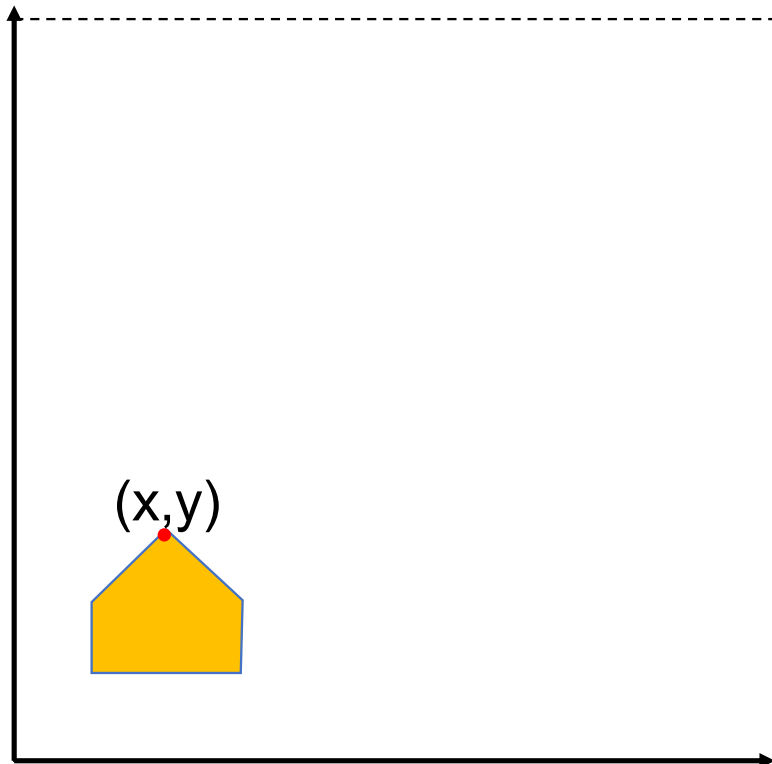
How to represent environment?

- Obstacle
 - Places where the robot cannot go
- Free space
 - Unoccupied space within the world
- Unknown
 - Robots “might” be able to go
 - There may be unknown obstacles or the state may be unreachable

Configuration Space

- The **configuration** of a robot: a specification of all points of a robot
- The **degrees of freedom** of a robot: is the smallest number of real-valued coordinates needed to represent its configuration
- **Configuration space** (C-space): is the n-dimensional space containing all possible configuration of a robot

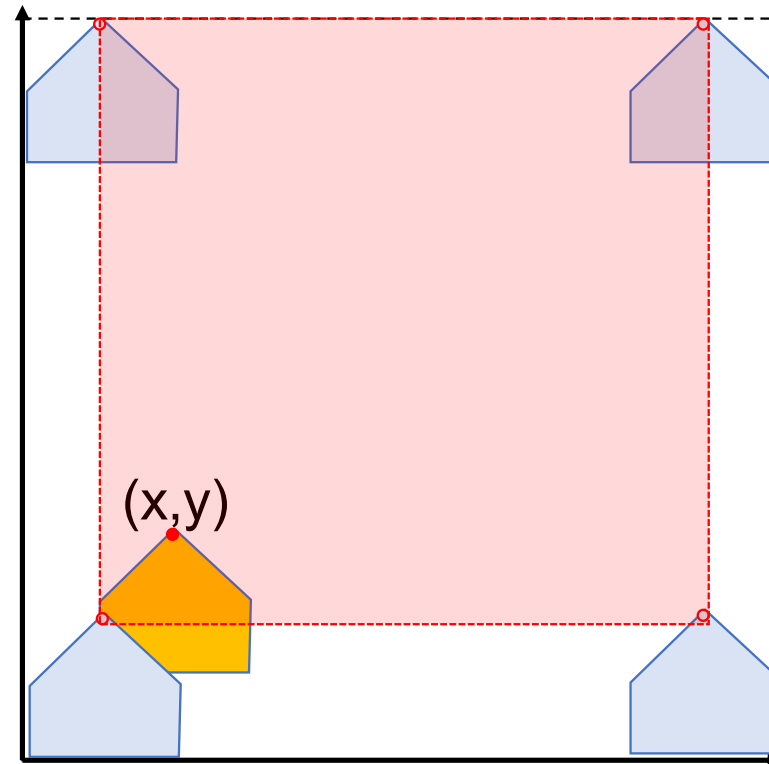
Example: Omni-directional 2D Robot



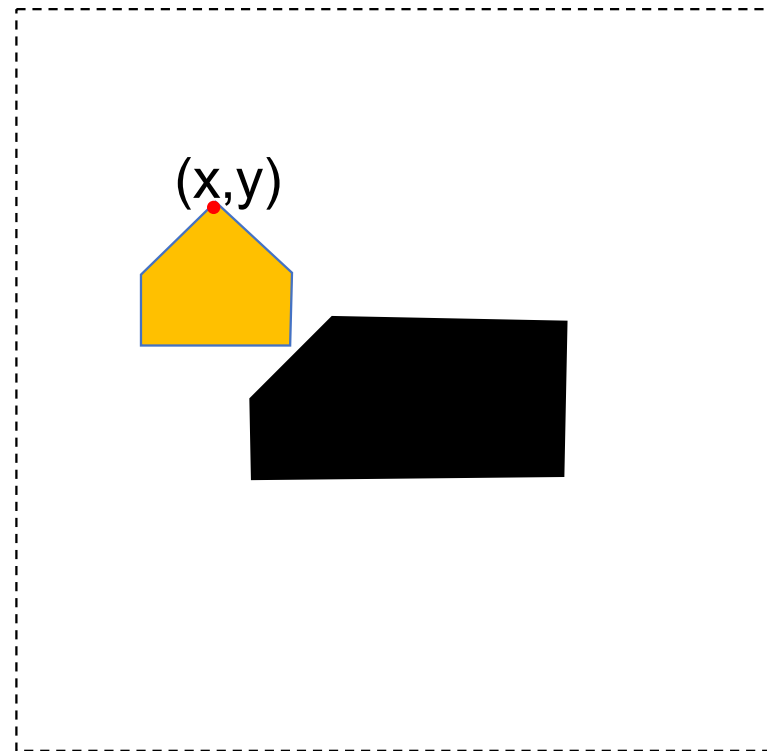
- Can only translate in the workspace, i.e., no rotation
 - Degree of freedom: 2
- The configuration is a point: (x, y)
 - we specify one point on a robot to define a robot's position
- What is the configuration space?

Configuration space of Omni-directional 2D Robot

Configuration space:
A n-dimensional space containing
all possible configuration of a robot

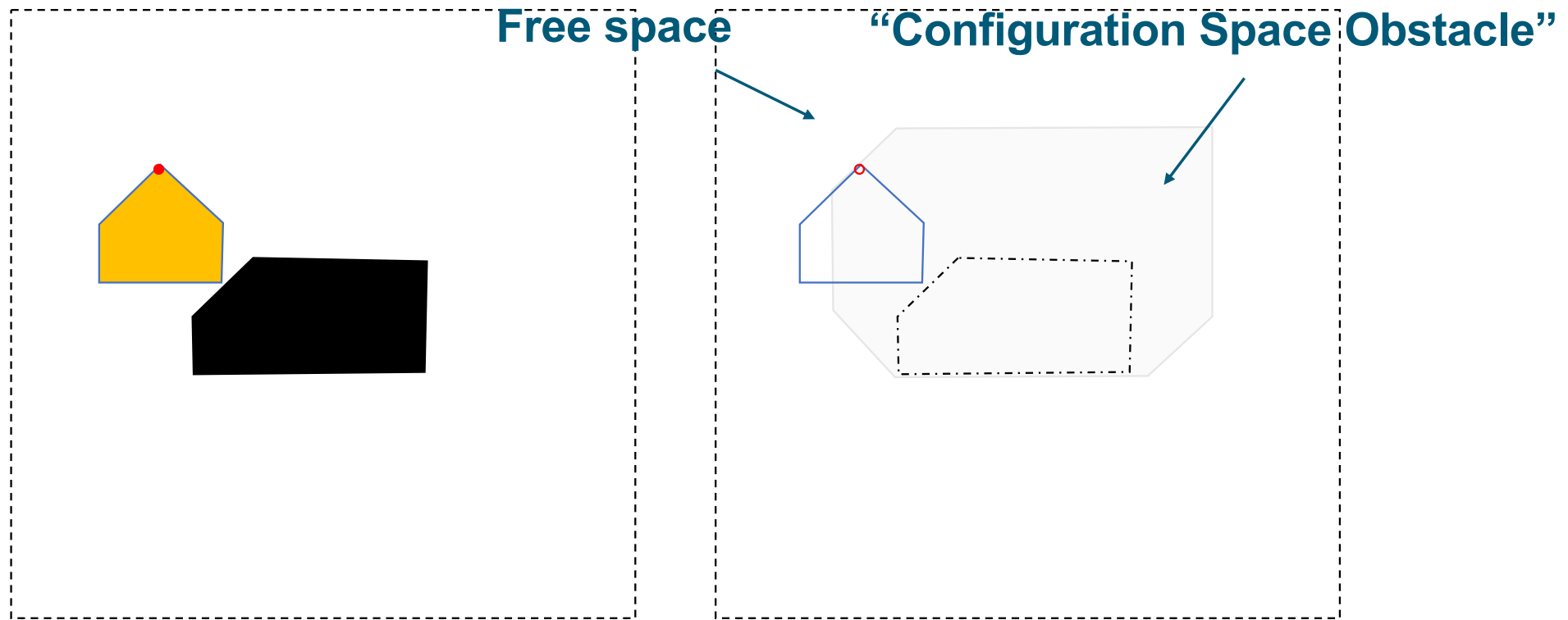


Obstacle



Obstacles make some of the original configuration space **no longer accessible**

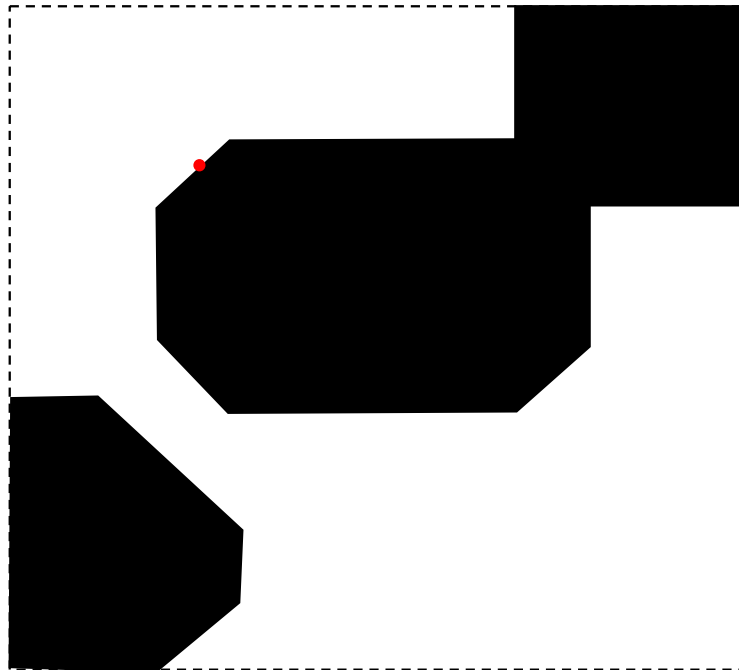
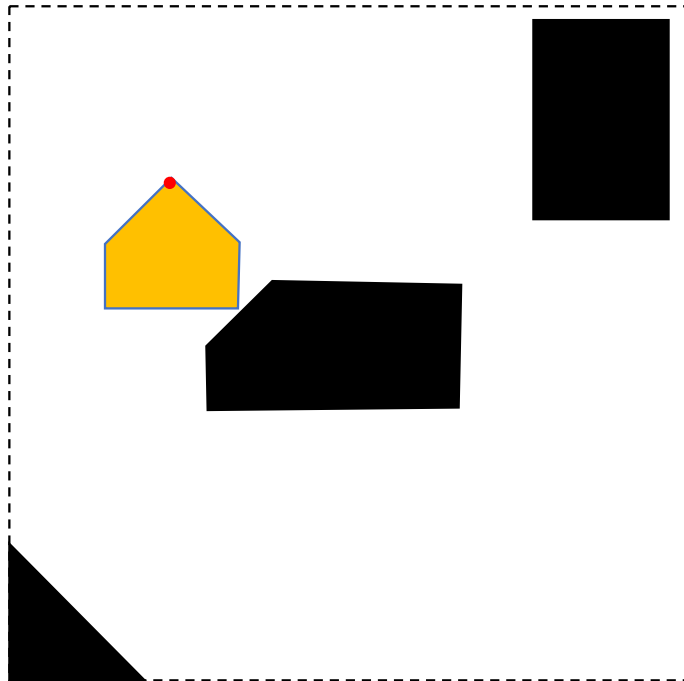
Configuration Space with Obstacles



Configuration space obstacle: x,y locations in the configuration space that the robot cannot reach because of the obstacle

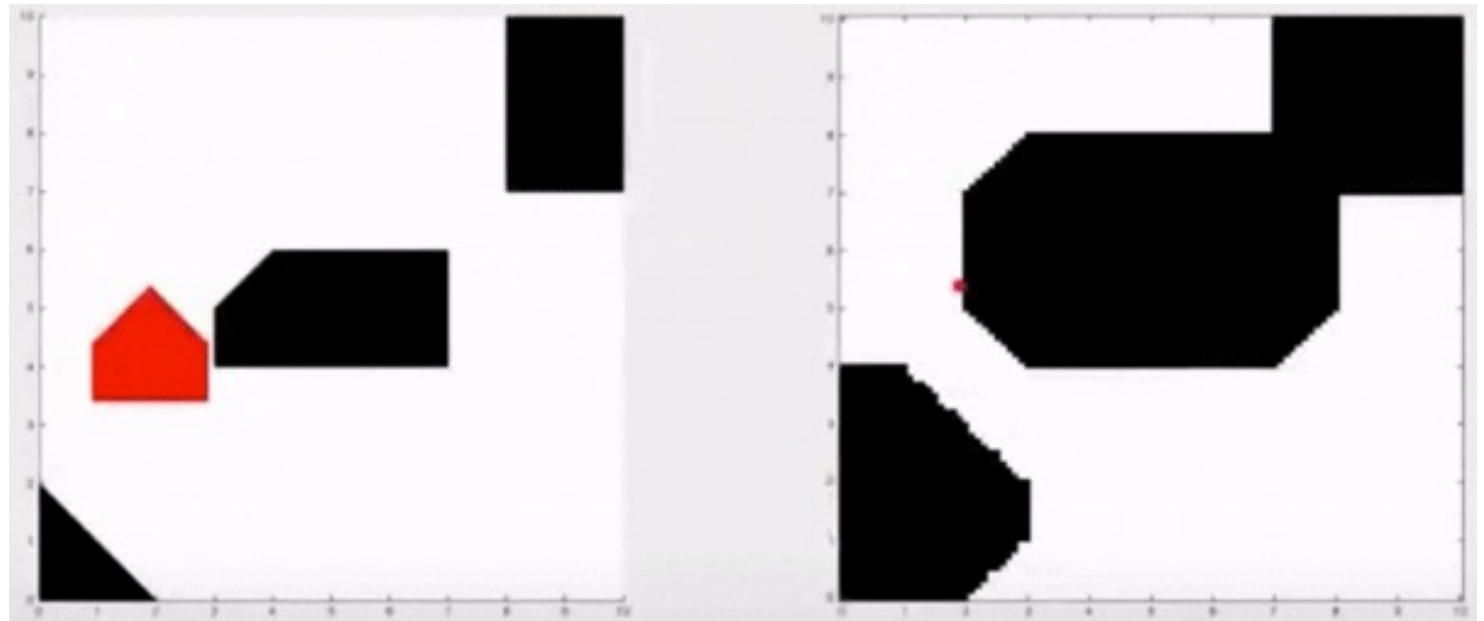
Multiple Obstacles?

Get the union of all C-space obstacle as the final C-Space obstacle

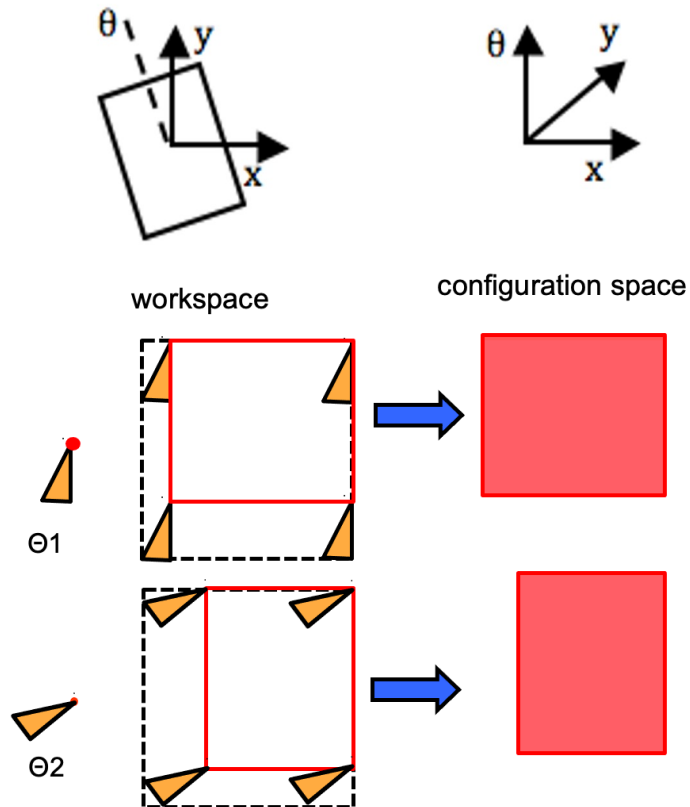


Path Planning in Configuration Space

- **Treat robot as a point in C-Space**
- Path planning = planning a trajectory in C-Space from starting configuration to ending configuration



Example: 2D Rigid Body

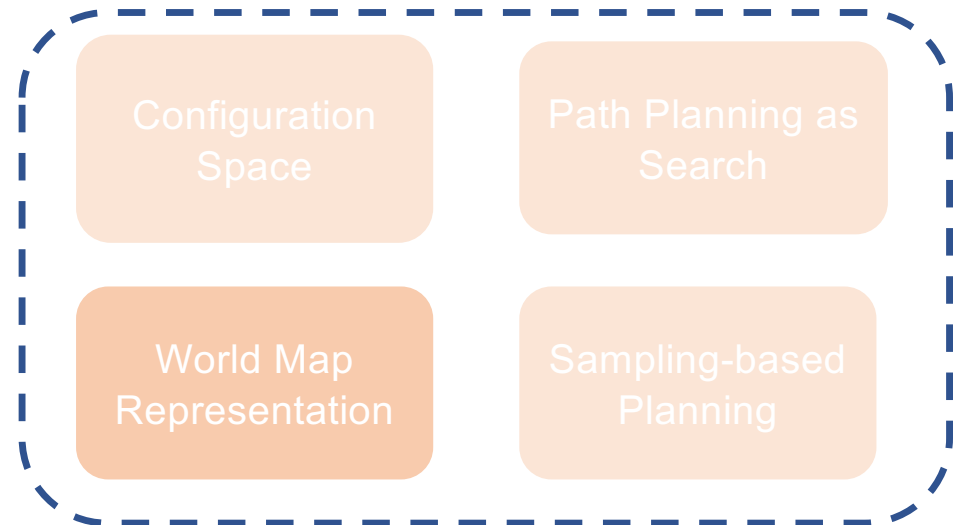
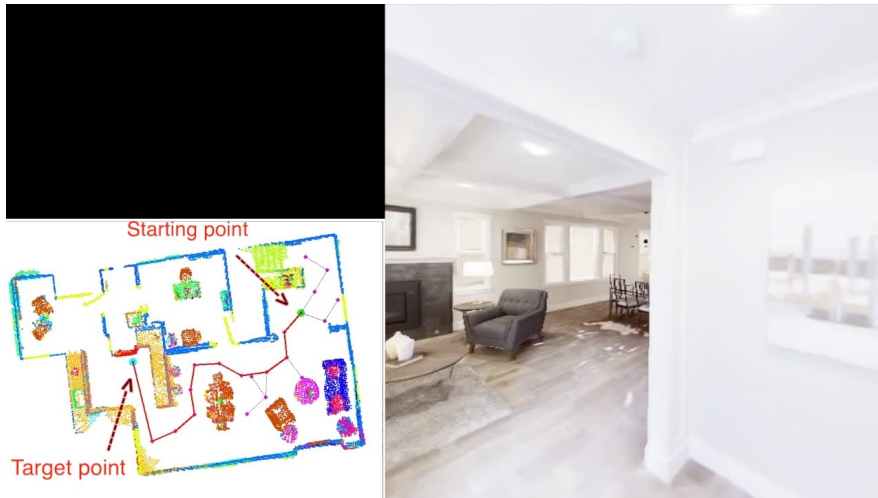


- Can only translate and rotate
 - Degree of freedom: 3
- The configuration is a point: (x, y, θ)
- What is the configuration space?

Configuration Space

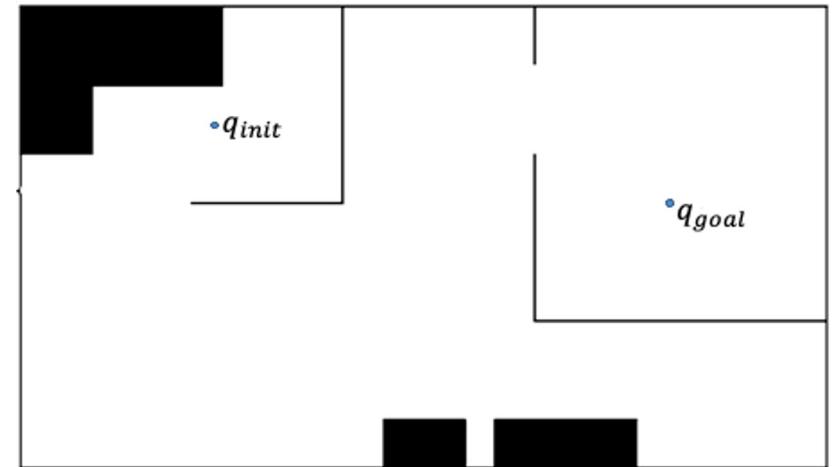
- The **configuration** of a robot: a specification of all points of a robot
- The **degrees of freedom** of a robot: is the smallest number of real-valued coordinates needed to represent its configuration
- **Configuration space** (C-space): is the n-dimensional space containing all possible configuration of a robot

Concept Map for Path Planning



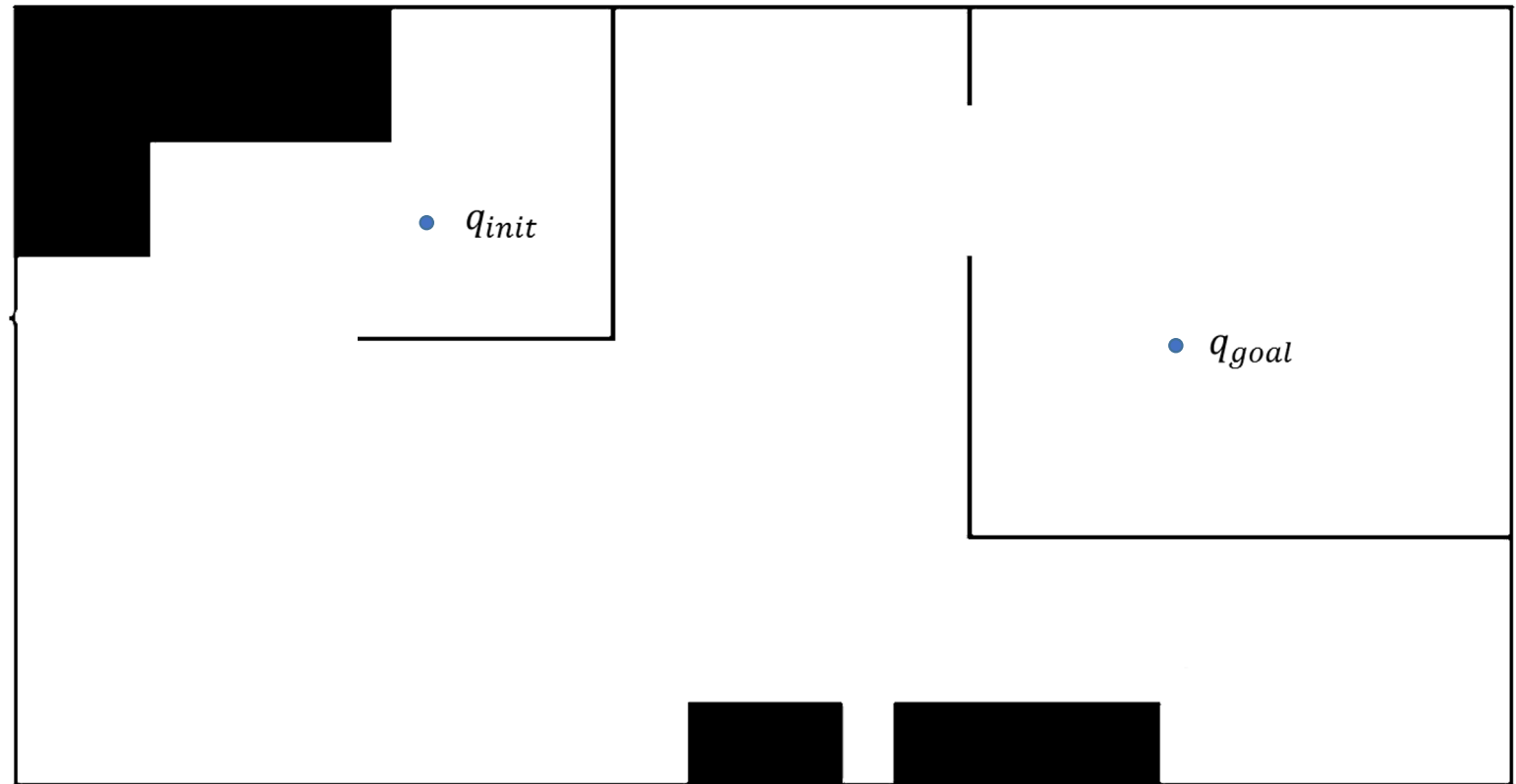
Path Planning in Configuration Space

- Assume we are given a world map, i.e., we know the location of free spaces and obstacles
- How to plan a path from q_{init} to q_{goal} ?



Quiz: How to obtain the world map?

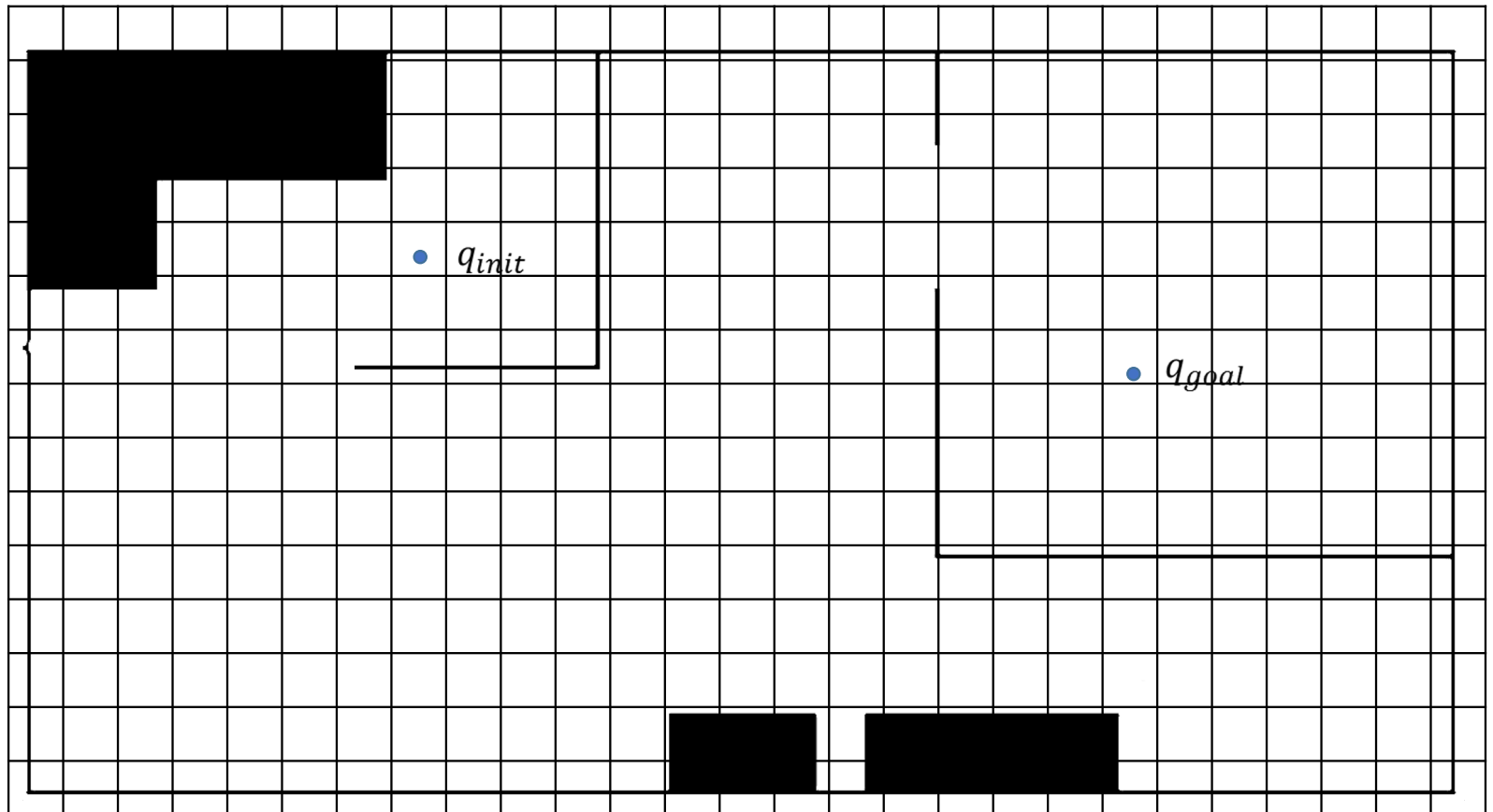
Environment



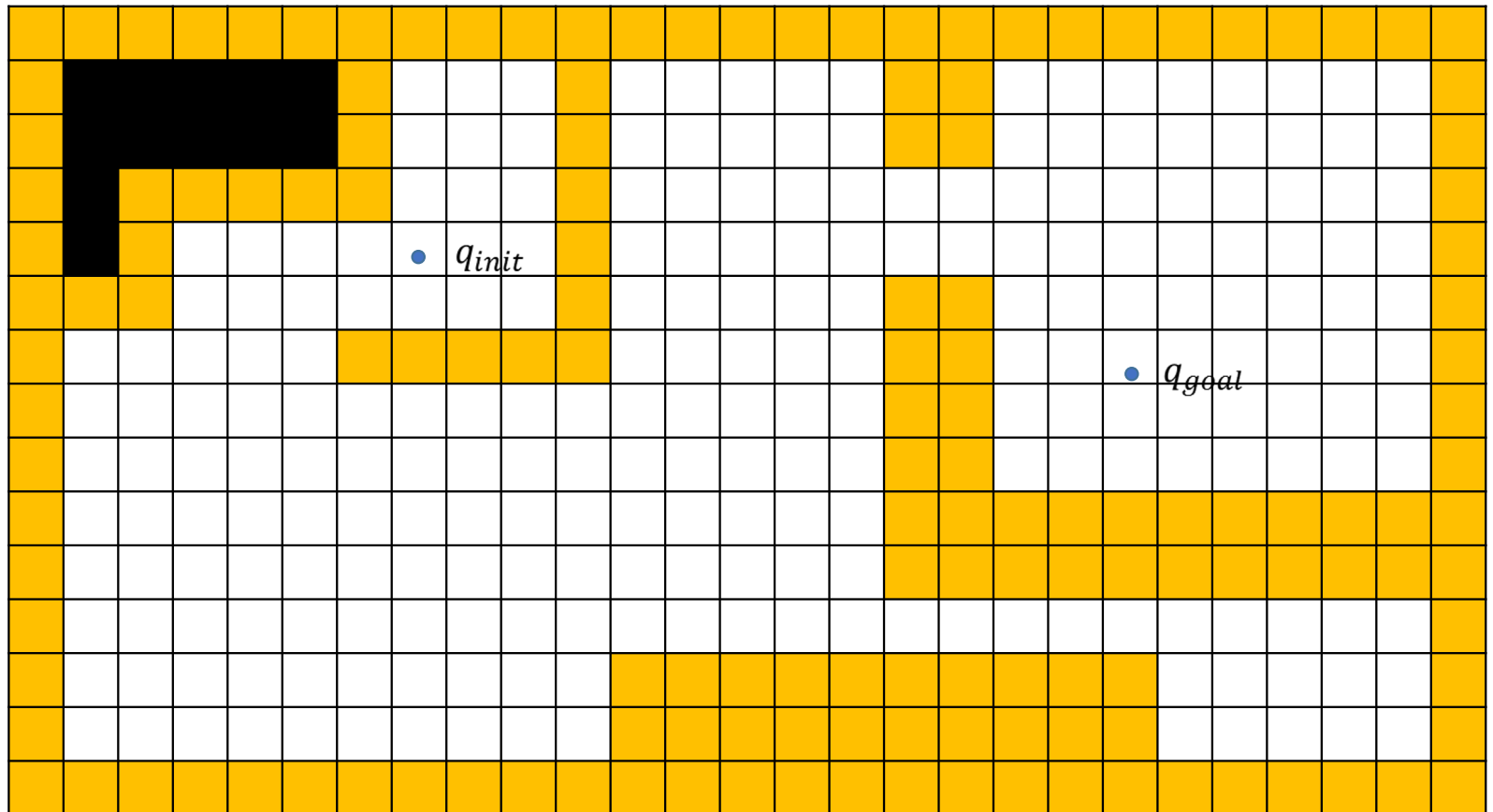
Representation: Grid

For example,
each grid has size of
50 cm by 50 cm

Where are the free
space and occupied
space?



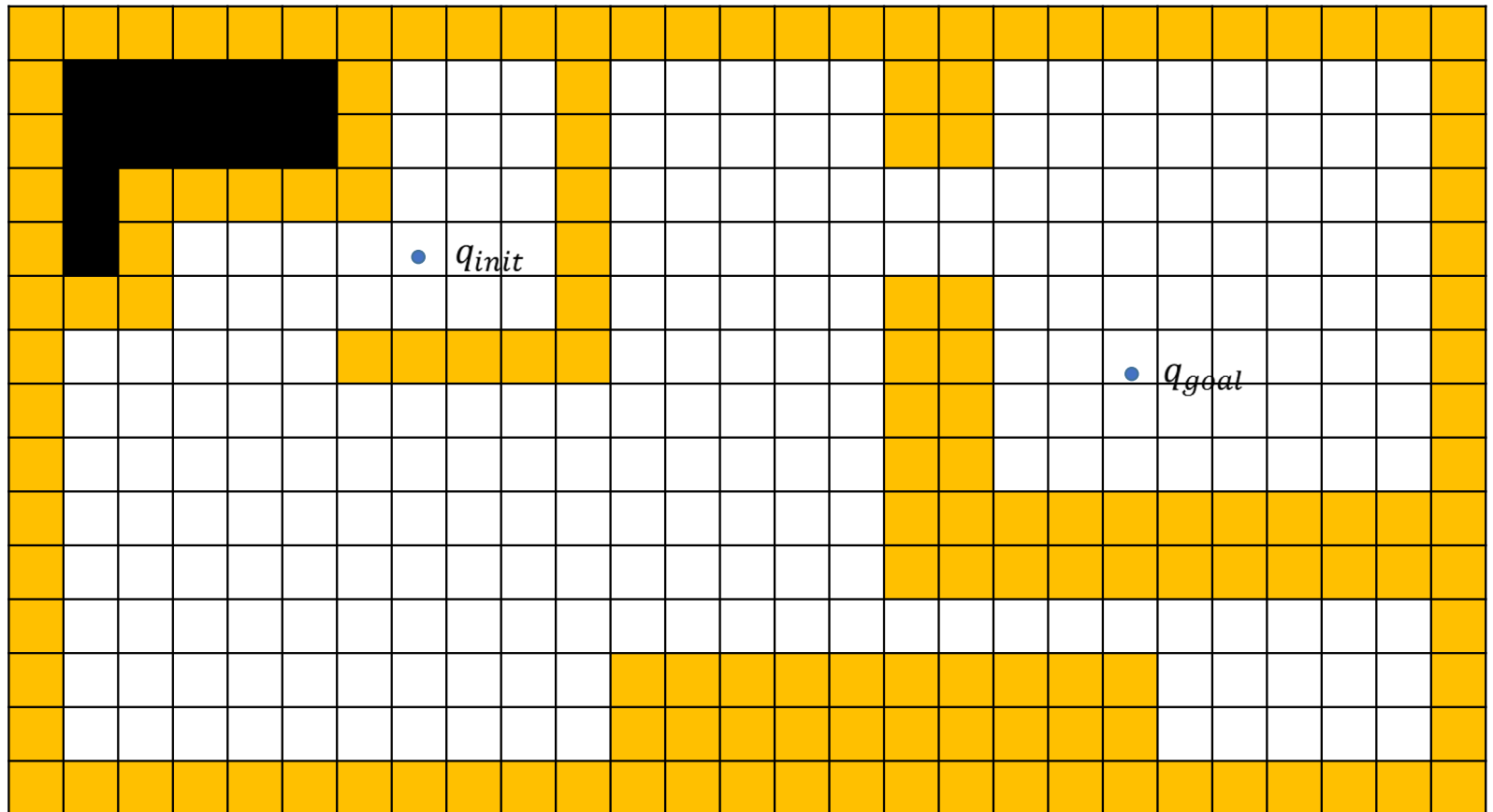
Representation: Occupancy Grid



What is robot's C-Space assume the robot has 2DoF?

- For example, each grid has size of 50 cm by 50 cm
- A robot is of size 40 cm by 40 cm

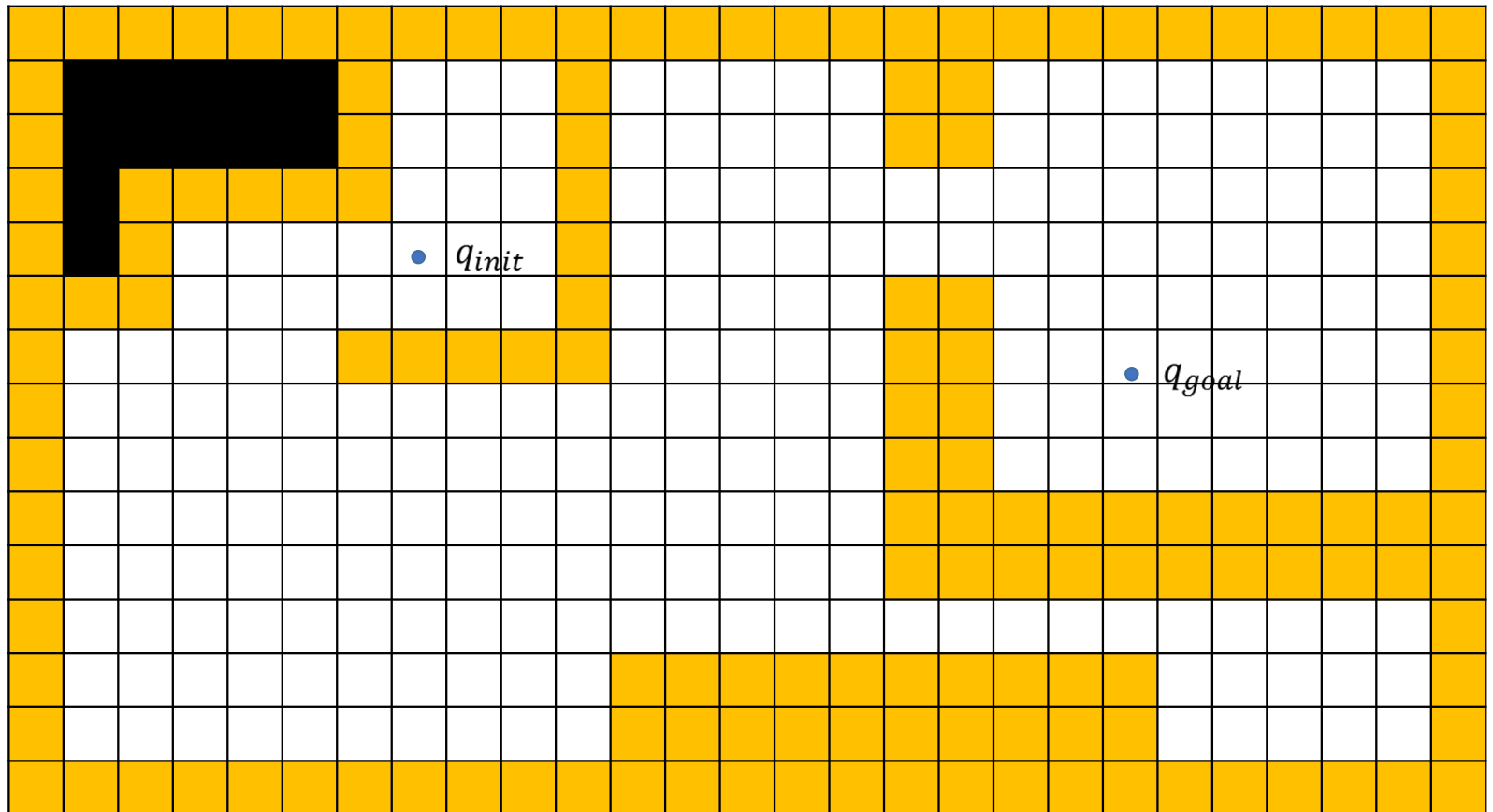
Representation: Occupancy Grid



What is robot's C-Space assume the robot has 2DoF?

- For example, each grid has size of 50 cm by 50 cm
- A robot is of size 40 cm by 40 cm

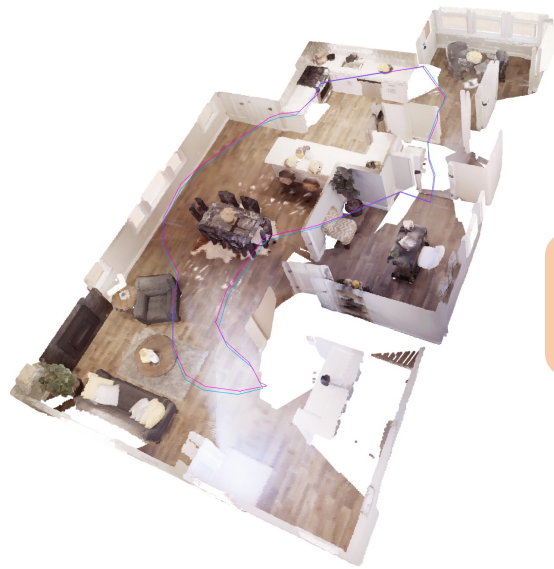
Representation: Occupancy Grid



What is robot's C-Space assume the robot has 2DoF?

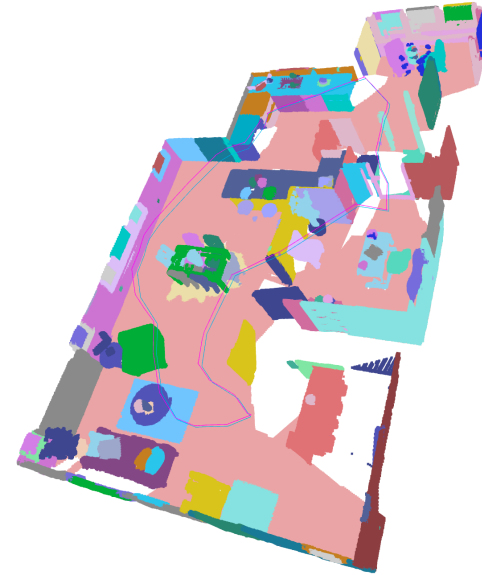
- For example, each grid has size of 50 cm by 50 cm
- A robot is of size 40 cm by 40 cm

Obstacle Detection



Point Cloud Map

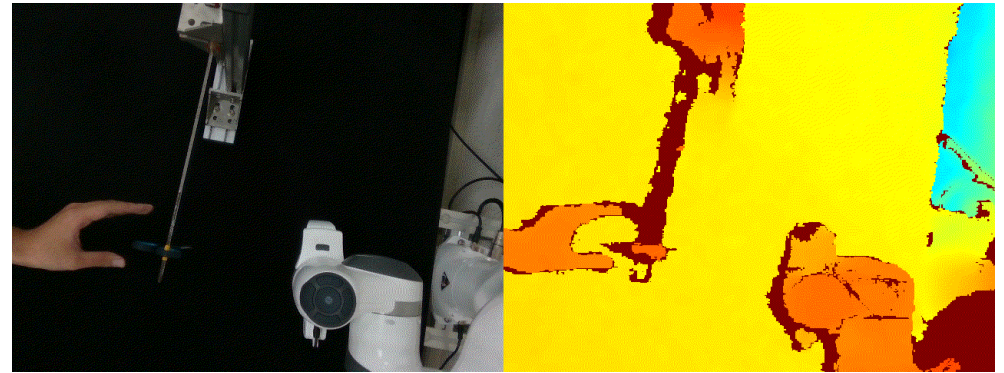
Semantic
Segmentation

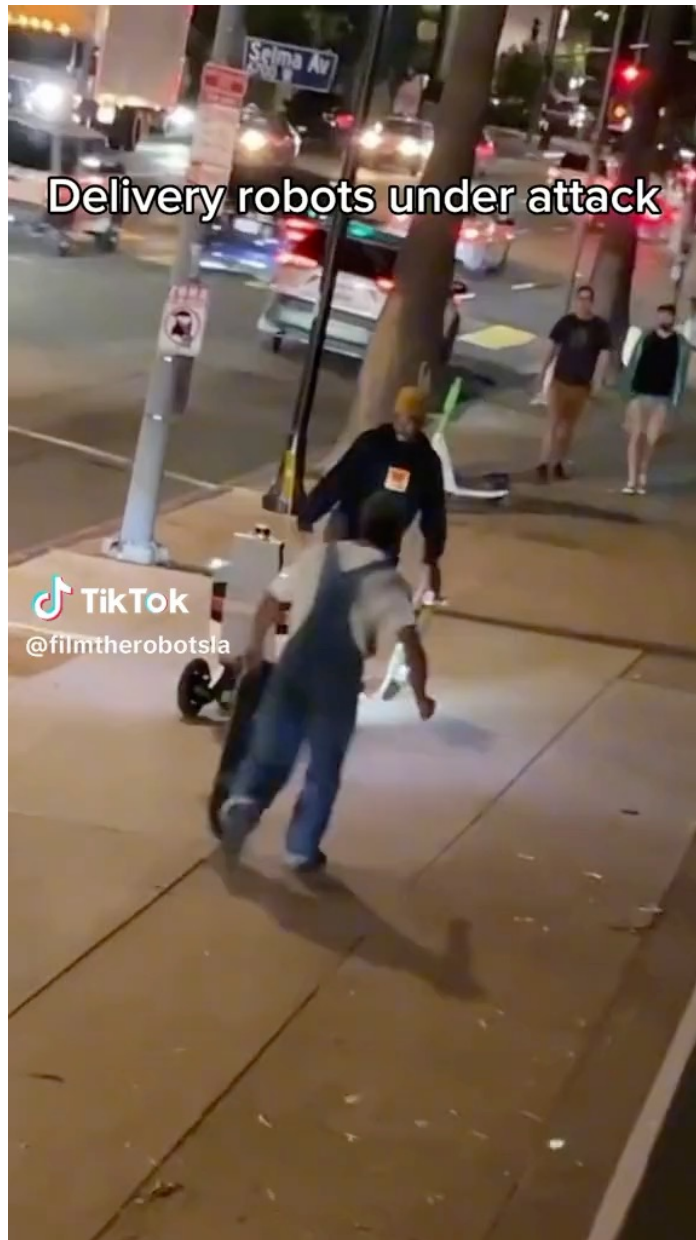


Semantic Map

Real-World Challenges

- Uncertainty (sensor noise)
- Dynamic obstacles
- Partial observability
- Robot dynamics





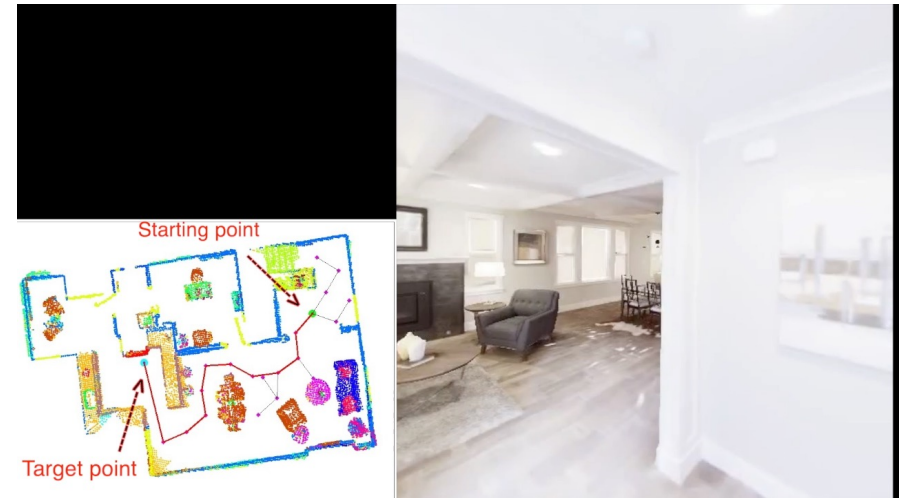
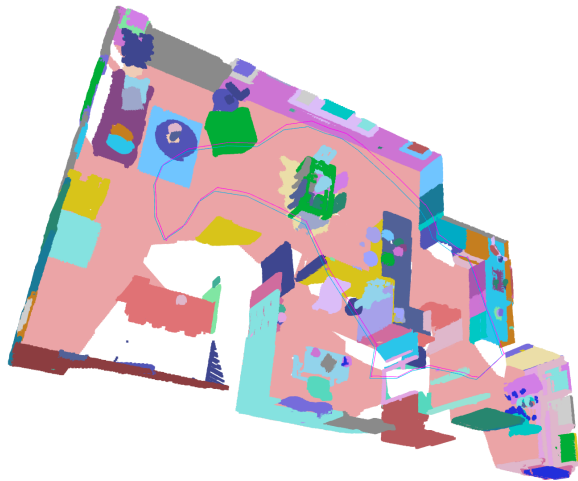
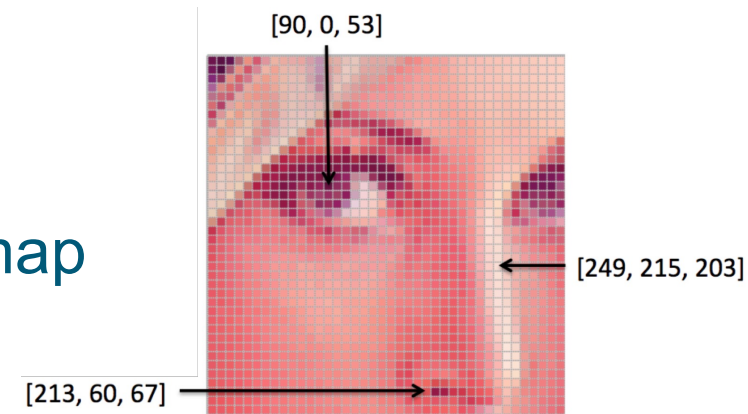
Delivery robots under attack

TikTok
@filmtherobotsla

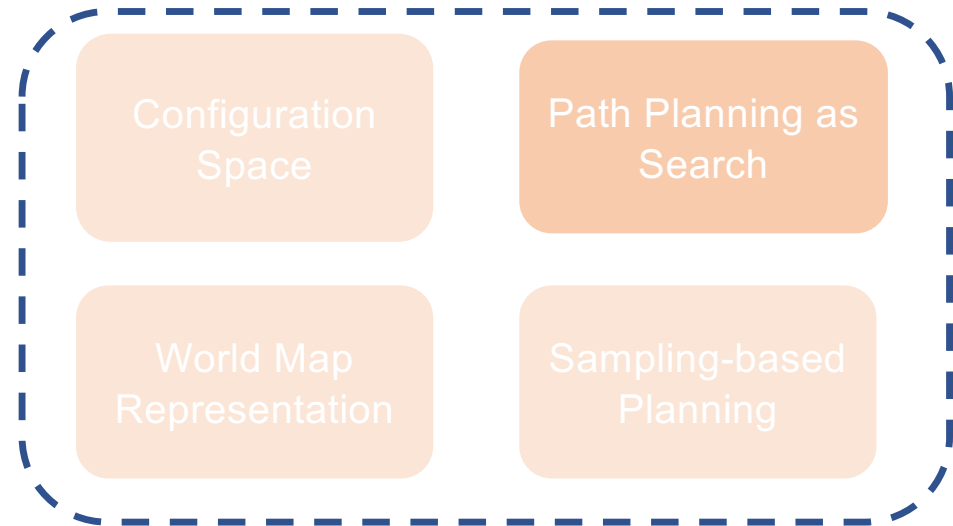
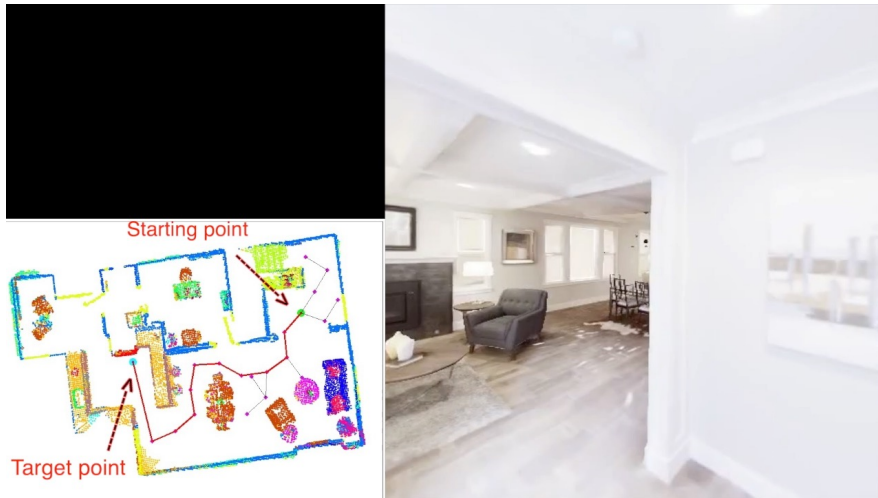
[https://www.tiktok.com/@filmtherobotsla/
video/7263179697486417195](https://www.tiktok.com/@filmtherobotsla/video/7263179697486417195)

HW3

- Semantic map (image) as occupancy grid map
 - Each pixel of a semantic map is a node
 - Adjacent nodes are connected via edges
- Apply path planning algorithms to generate a sequence of actions from start to goal positions



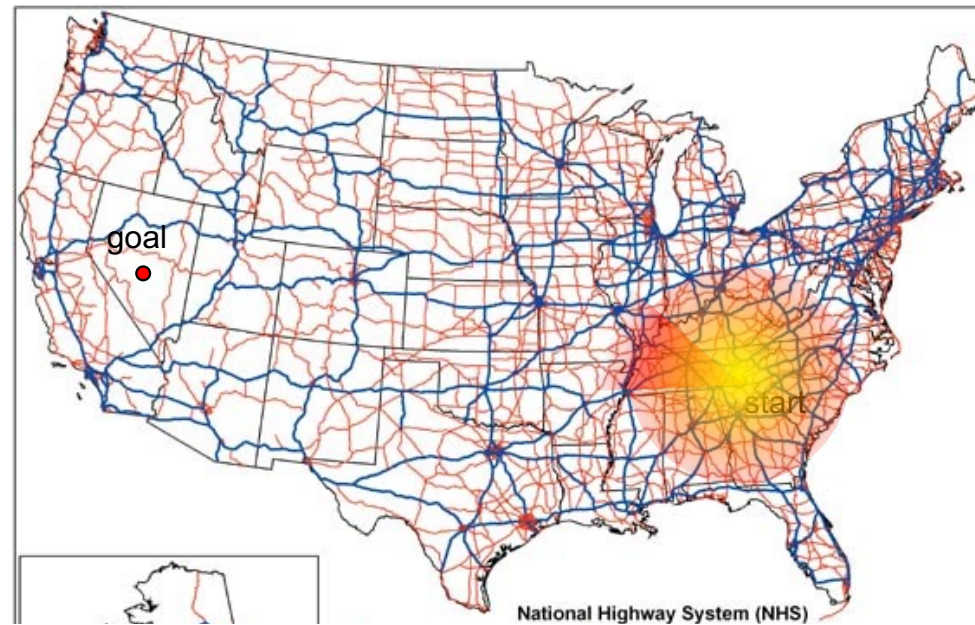
Concept Map for Path Planning



Path Planning as Search

- Task: find a path from start vertex to goal vertex given a graph
- Search Problem:
 - State (configurations of the world)
 - Initial state and goal test
 - Action (go to the adjacent node, e.g., NSEW in a grid-like map)
 - Step and path costs (e.g., 1)
 - Transition model, also called Successor (i.e., $S_{t+1} = T(S_t, A_t)$)
- A solution is a sequence of actions (a plan) which transforms the initial state to a goal state

Uninformed vs. Informed Search



Uninformed search



Informed search

Search Algorithms

- **Uninformed search algorithms**
 - Can only generate successors and distinguish goals from non-goals
 - Breadth-first search (BFS)
 - Depth-first search (DFS)
 - Uniform Cost Search (UCS)
- **Informed search algorithms**
 - Strategy that knows whether one non-goal is more promising than another
 - A* Search

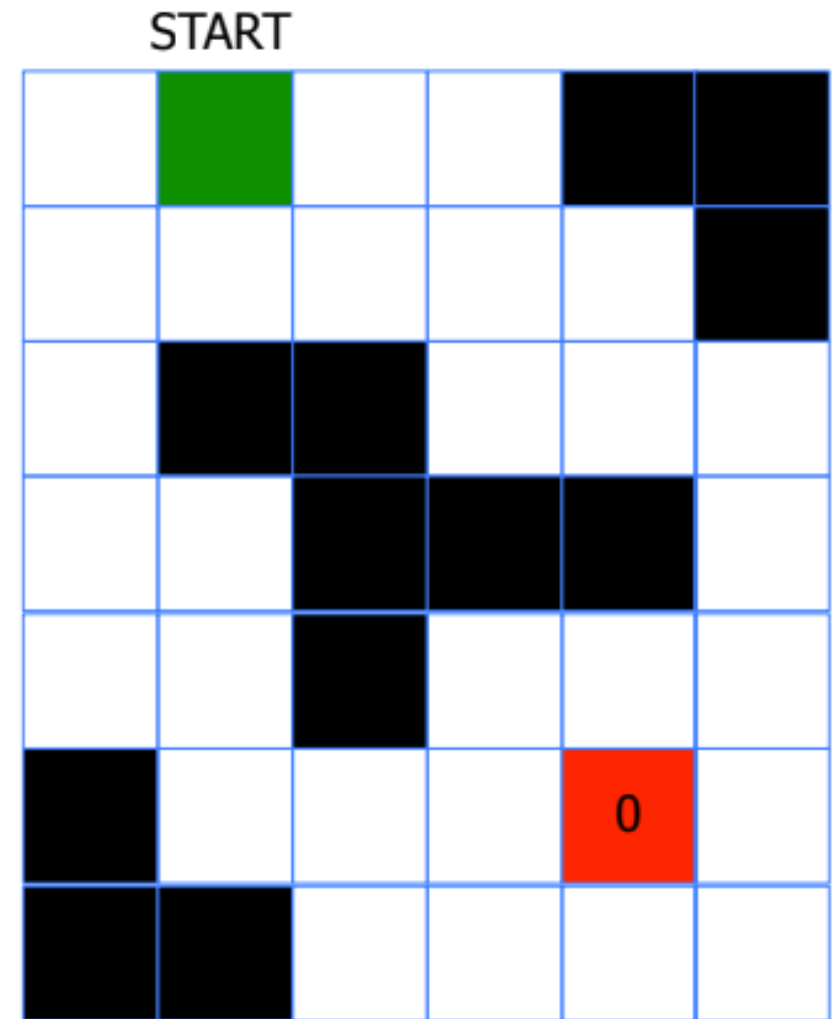
Planning on a Grid

Grassfire Algorithm (Uninformed search):

A path-planning algorithm that simulates fire spreading from a destination cell through a grid, labeling cells with their distance from that destination..

BFS from the destination

Begin by marking the destination node with a distance value of 0

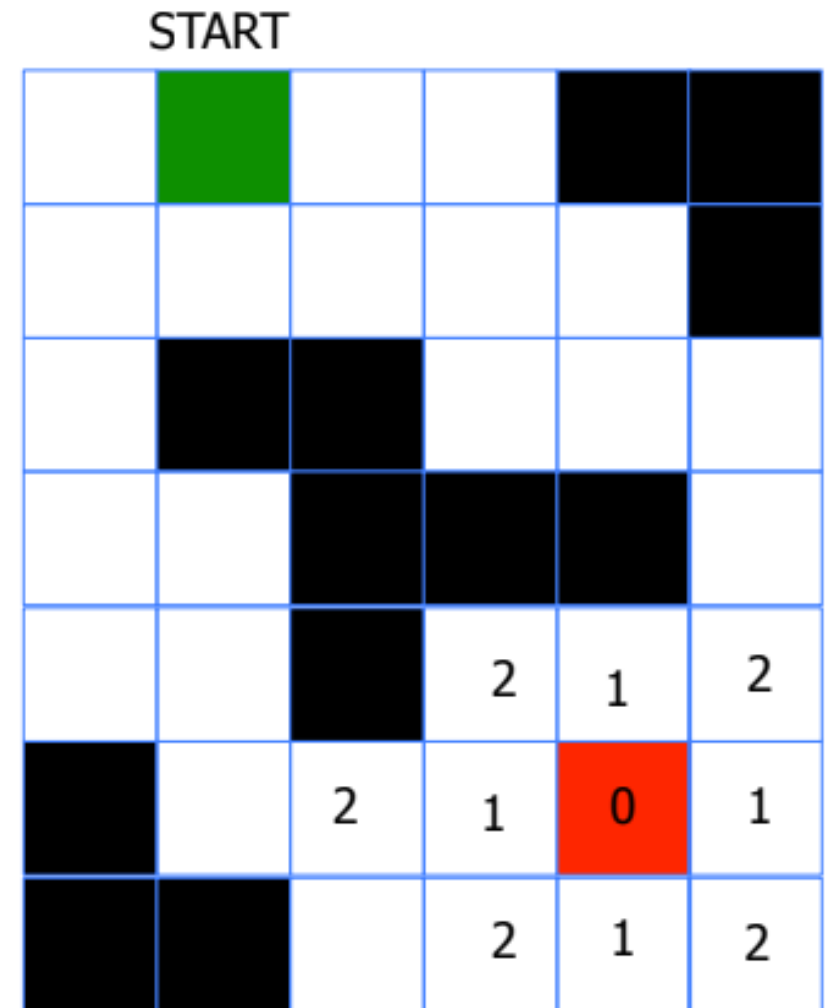


Planning on a Grid

Grassfire Algorithm (Uninformed search):

Begin by marking the destination node with a distance value of 0

Continue to other adjacent grid cells



Planning on a Grid

Grassfire Algorithm (Uninformed search):

Begin by marking the destination node with a distance value of 0

Continue to other adjacent grid cells

On every iteration the marking radiates outward from the destination like a fire spreading

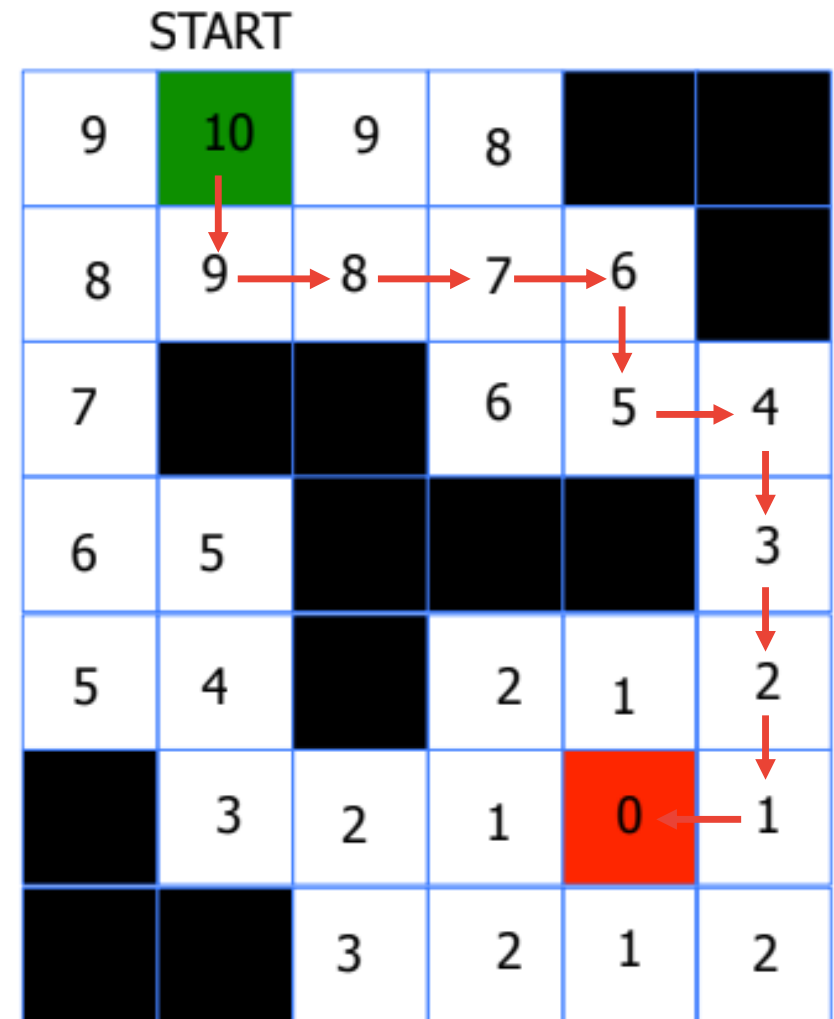


Planning on a Grid

Grassfire Algorithm (Uninformed search):

The distance values produced by the algorithm indicate the smallest number of steps needed to move from each node to the goal

To move towards the destination from any node, move towards the neighbor with the smallest distance value

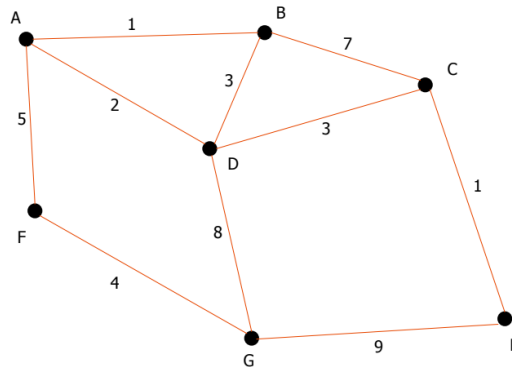
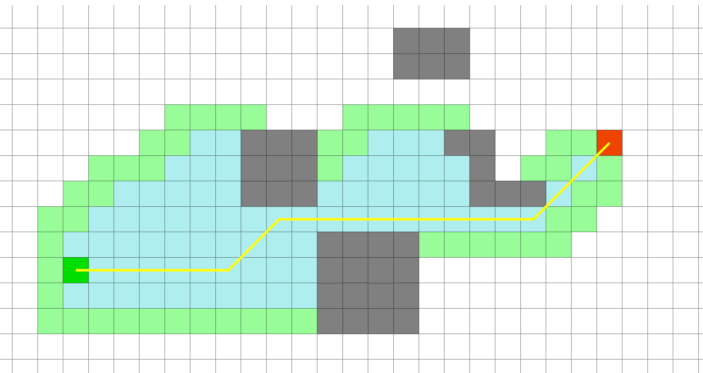


A* Search (Informed Search)

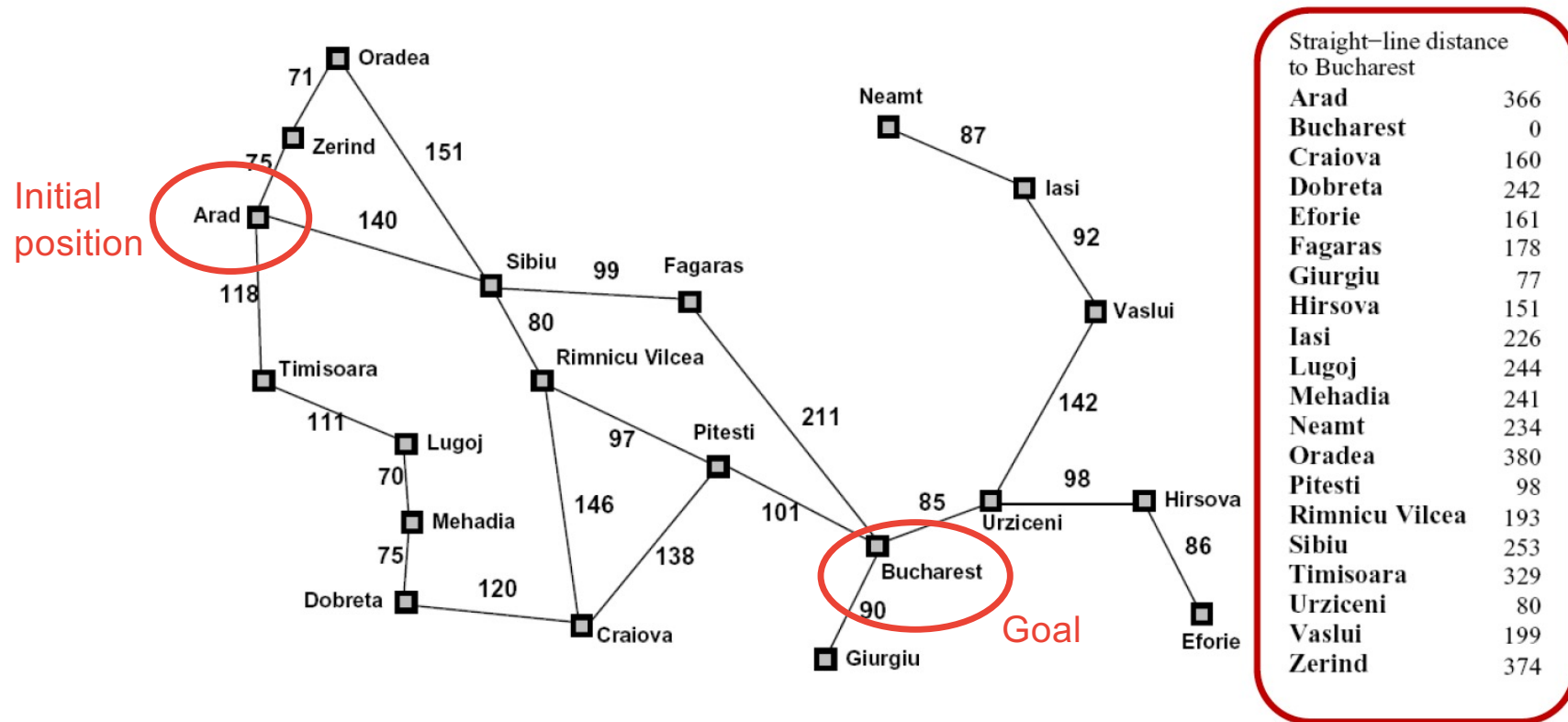
- A* search attempts to improve search performance by incorporating a **heuristic function** that guides the path planner
- Heuristic functions are used to map every node in the graph to a non-negative value, that represents the “best guess distance” from that node to the goal
 - for example, Euclidean distance (L2 distance)

Google Map Navigation

- How to turn the problem into a graph-like structure?
- What would be a good heuristic function?

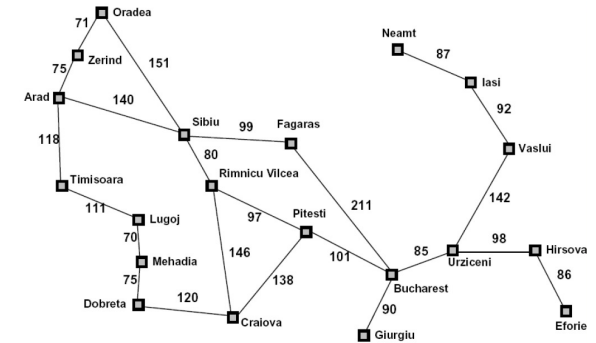


A* Search

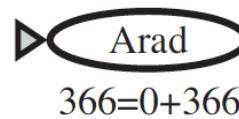


A* Search

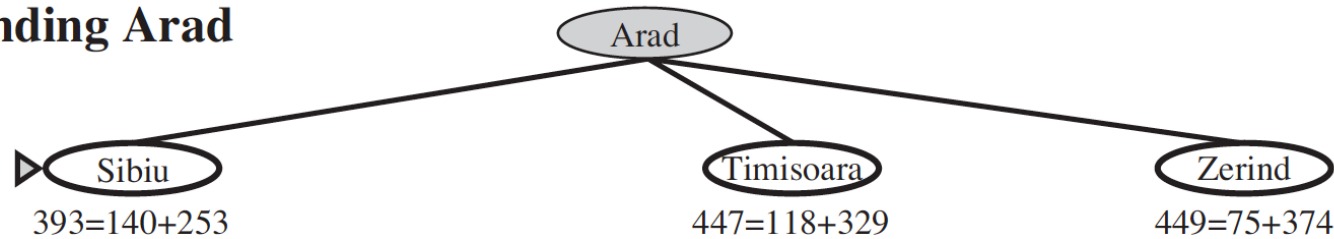
path cost + Heuristics



(a) The initial state



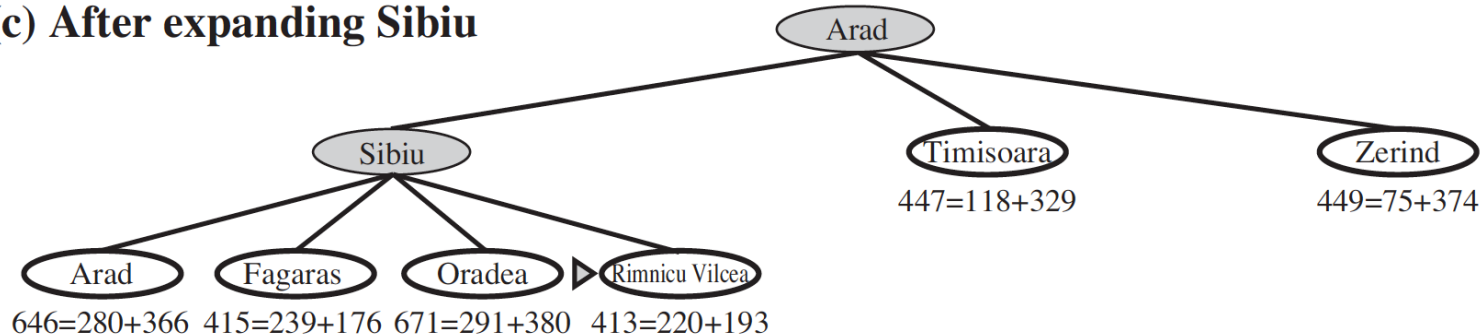
(b) After expanding Arad



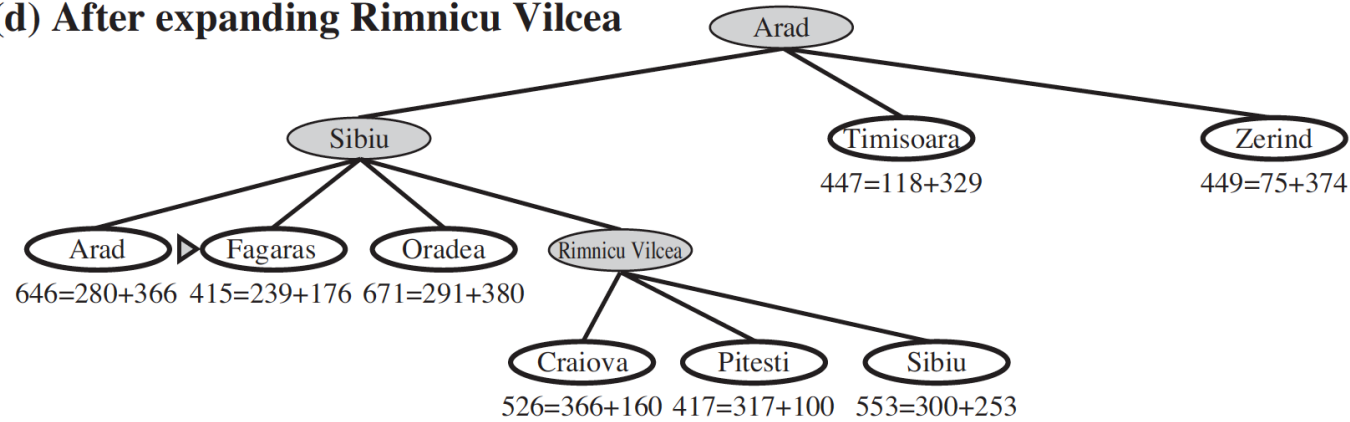
Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

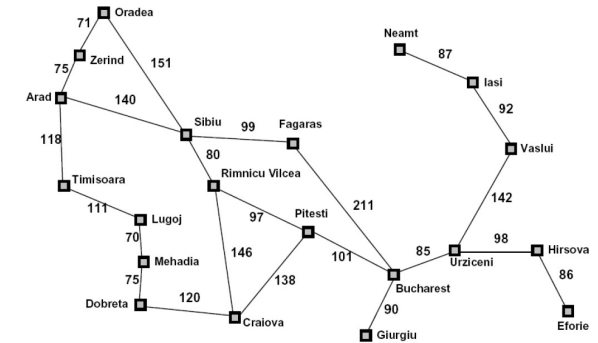
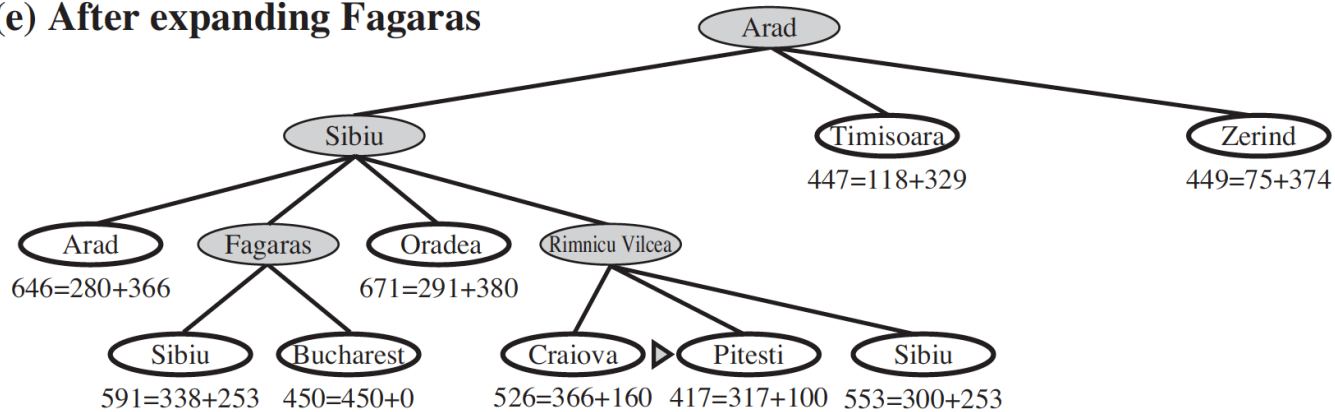
(c) After expanding Sibiu



(d) After expanding Rimnicu Vilcea



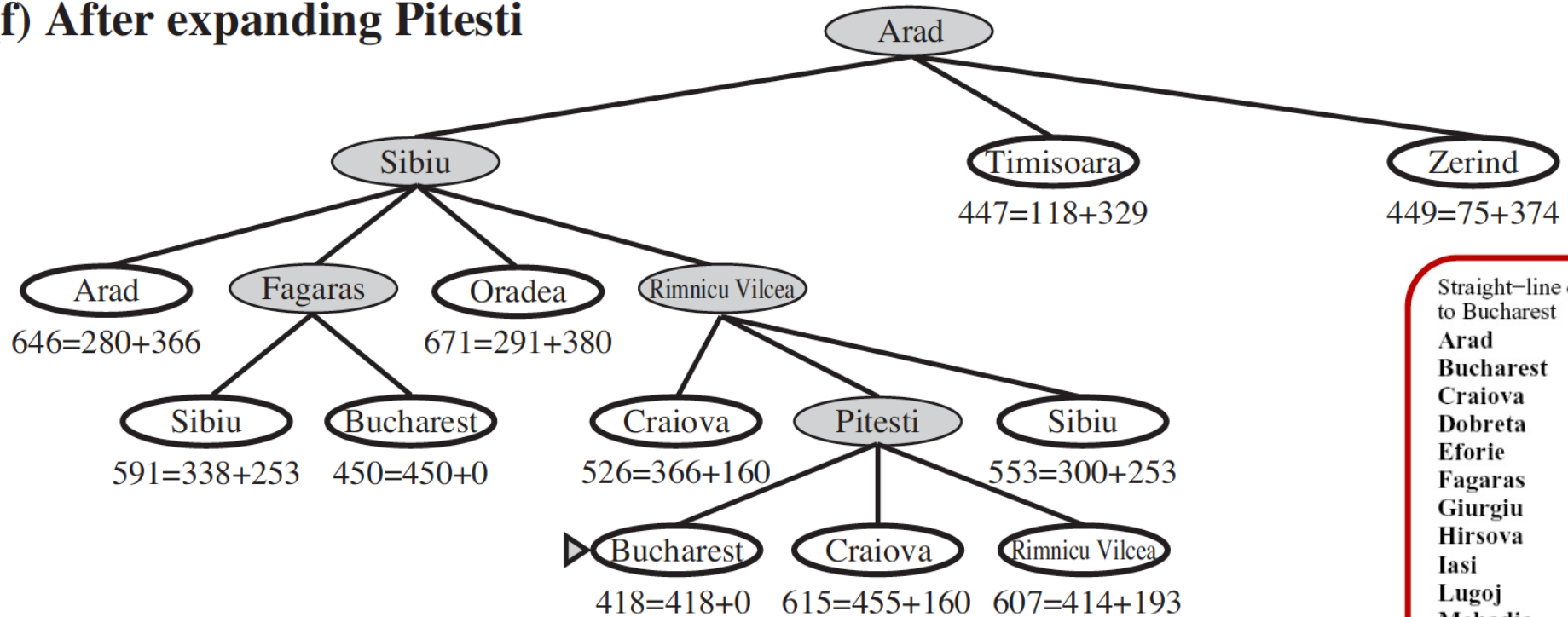
(e) After expanding Fagaras



Straight-line distance to Bucharest

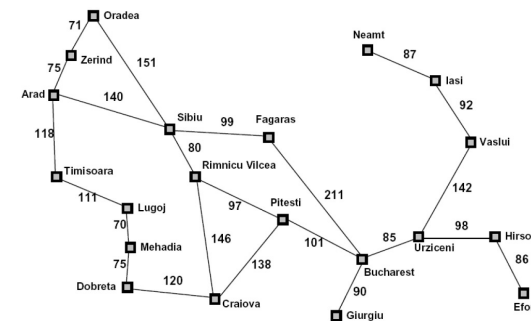
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

(f) After expanding Pitesti



Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

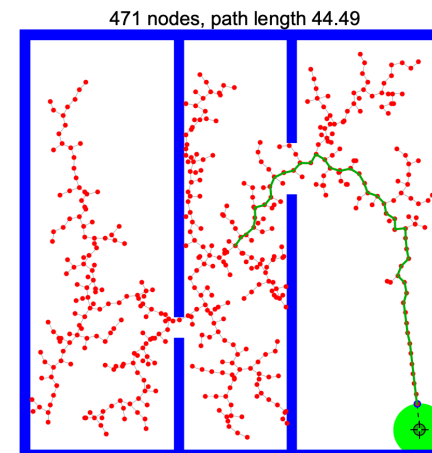
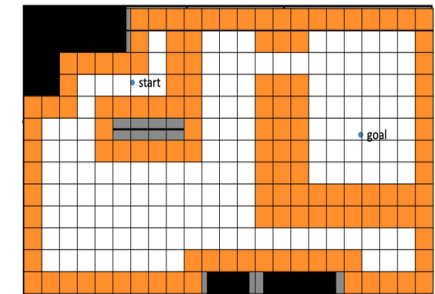
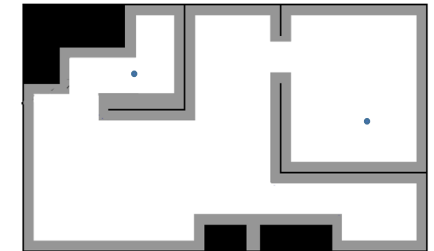
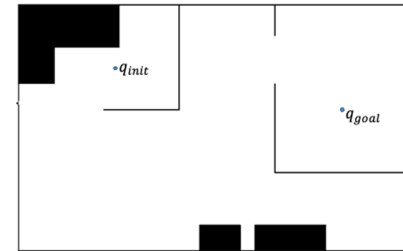


Live Demo

- <https://qiao.github.io/PathFinding.js/visual/>

Summary

- Configuration space with obstacles
 - Robot as a point
- Turn configuration space into a graph
 - Occupancy grid
- Path planning as search
 - Uninformed or informed search (A*)
- Sample based path planning
 - PRM, RRT, RRT*

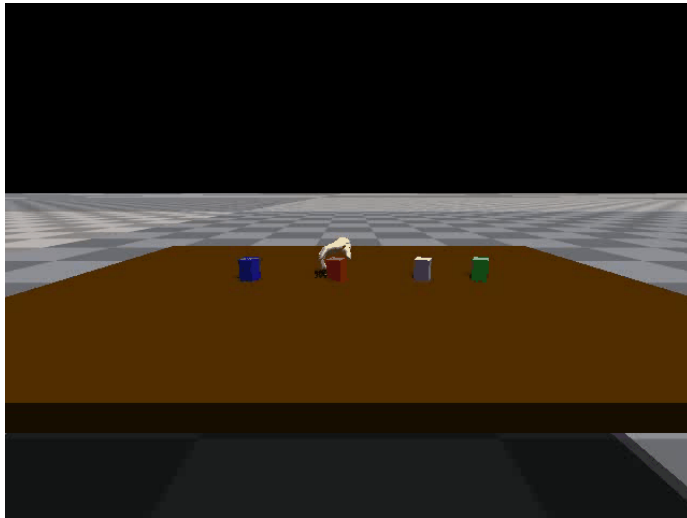


START

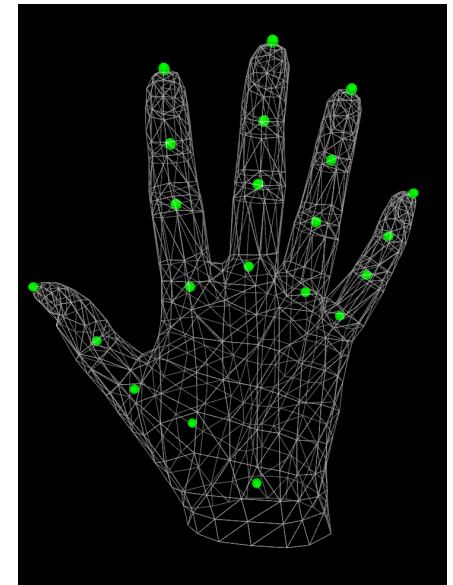
9	10	9	8		
8	9	8	7	6	
7			6	5	4
6	5				3
5	4		2	1	2
	3	2	1	0	1
		3	2	1	2

Limitation of Grid-based Methods

- Running time increases exponentially with the dimension of the configuration space
 - For example, dexterous hand with 24 DoF

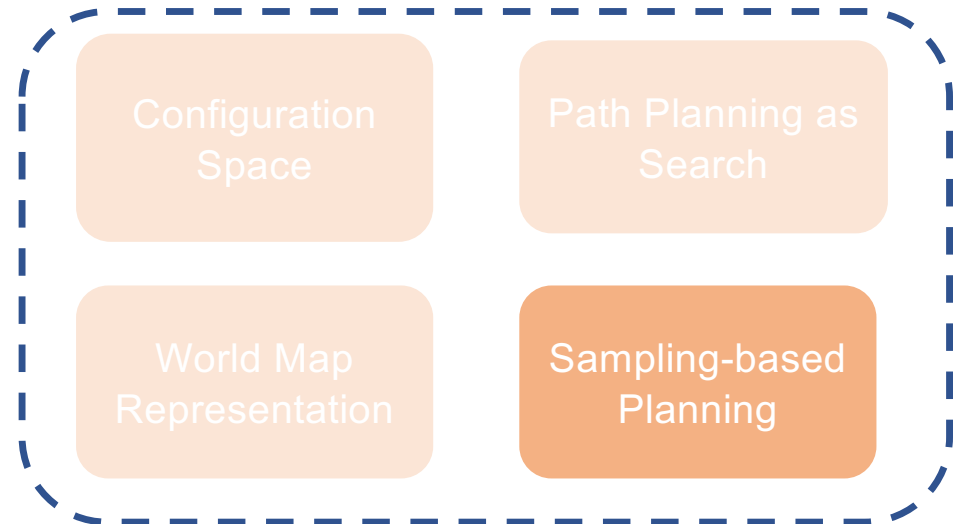
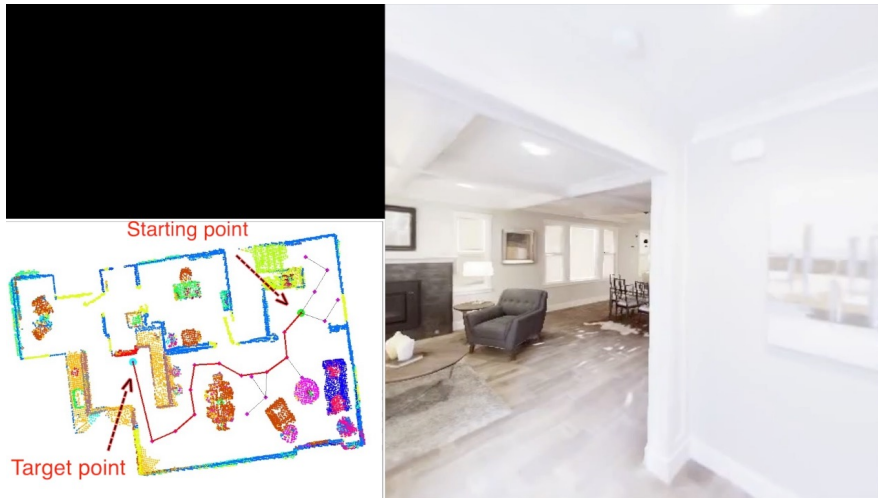


Credit: Han Pan



<https://github.com/otaheri/MANO>

Concept Map for Path Planning



Sampling based Planning

- Key idea:
 - Rather than exhaustively explore ALL possibilities, **randomly** explore a smaller subset of possibilities while keeping track of progress
- Search for a collision-free path only by sampling points

Probability Roadmap (PRM)

- Probabilistic Roadmap methods proceed in two phases :
 - Preprocessing Phase – construct a roadmap G
 - Query (Search) Phase –search a path from q_{init} and q_{goal}
- The roadmap is an undirected graph $G = (N, E)$
 - The nodes in N are a subset of configurations of the robot sampled that are collision free
 - The edges in E correspond to feasible straight-line paths

Probability Roadmap (PRM) – Phase 1



Build-Roadmap:

G.init() ; i = 0

While i < N

$\alpha \leftarrow \text{SAMPLE}$

if $\alpha \in C_{free}$

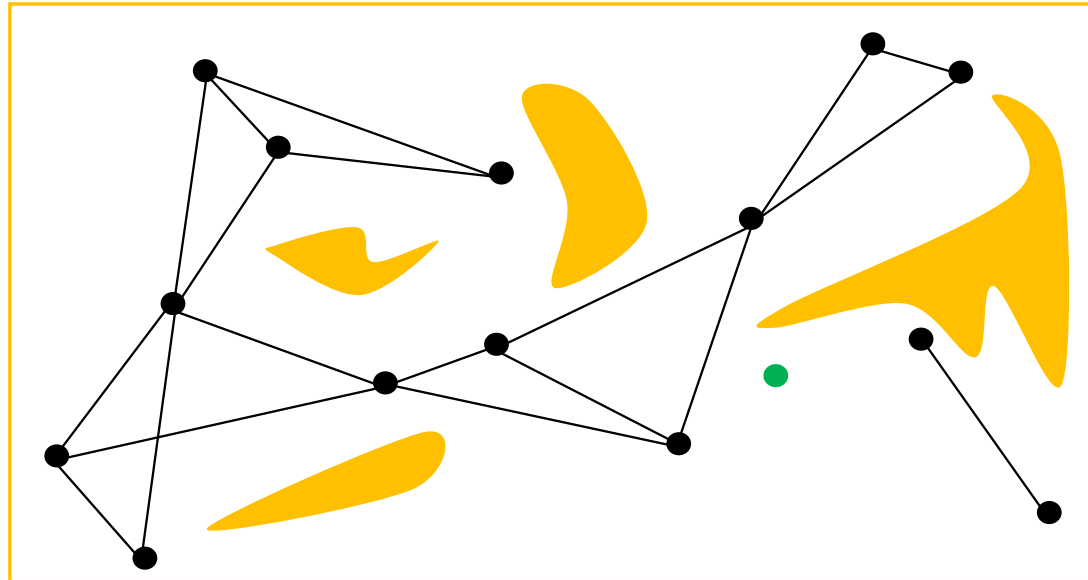
G.add Vertex(α); i = i+1

for each $q \in \text{ngd}(\alpha, G)$

if (connect(α , G))

G.addEdge(α , q)

Probability Roadmap (PRM) – Phase 1



Build-Roadmap:

G.init() ; i = 0

While i < N

$\alpha \leftarrow \text{SAMPLE}$

if $\alpha \in C_{\text{free}}$

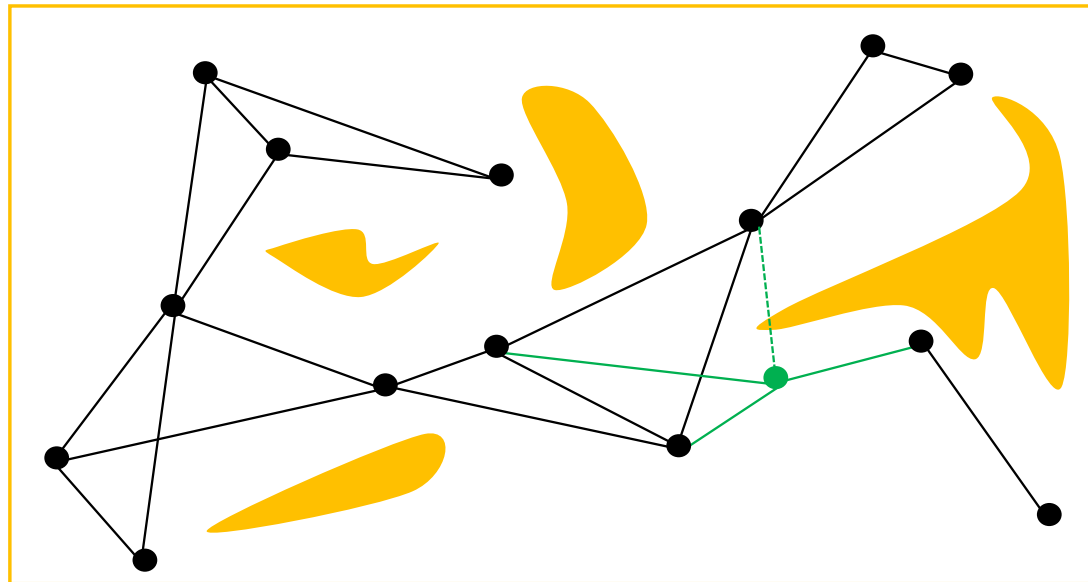
G.add Vertex(α); i = i+1

for each $q \in \text{ngd}(\alpha, G)$

if (connect(α , G))

G.addEdge(α , q)

Probability Roadmap (PRM) – Phase 1



Build-Roadmap:

$G.init()$; $i = 0$

While $i < N$

$\alpha \leftarrow \text{SAMPLE}$

if $\alpha \in C_{free}$

$G.add\ Vertex(\alpha)$; $i = i+1$

for each $q \in ngd(\alpha, G)$

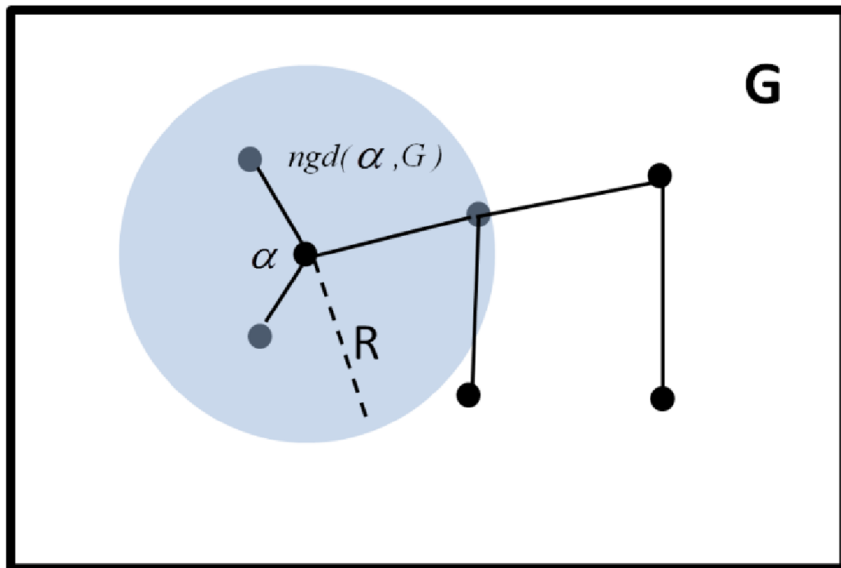
if (connect(α , G))

$G.addEdge(\alpha, q)$

K nearest
neighbor

Collision
Detection

Connecting a New Sample



- Generate randomly
- The distance of neighbor R is the number that tells you how far you think you can connect to configurations by straight lines.

Distance Function

- Measure the distance between two points in **configuration space**

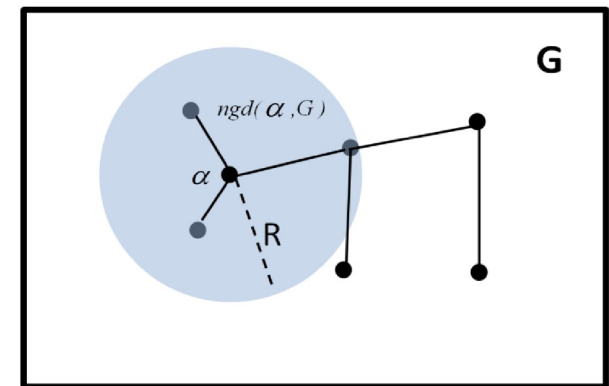
- Common choice in 2 DoF:

- The L1 distance : $Dist_1 = \sum_i |\mathbf{x}_i - \mathbf{y}_i|$
- The L2 distance : $Dist_2 = \sqrt{(\sum_i (\mathbf{x}_i - \mathbf{y}_i)^2)}$

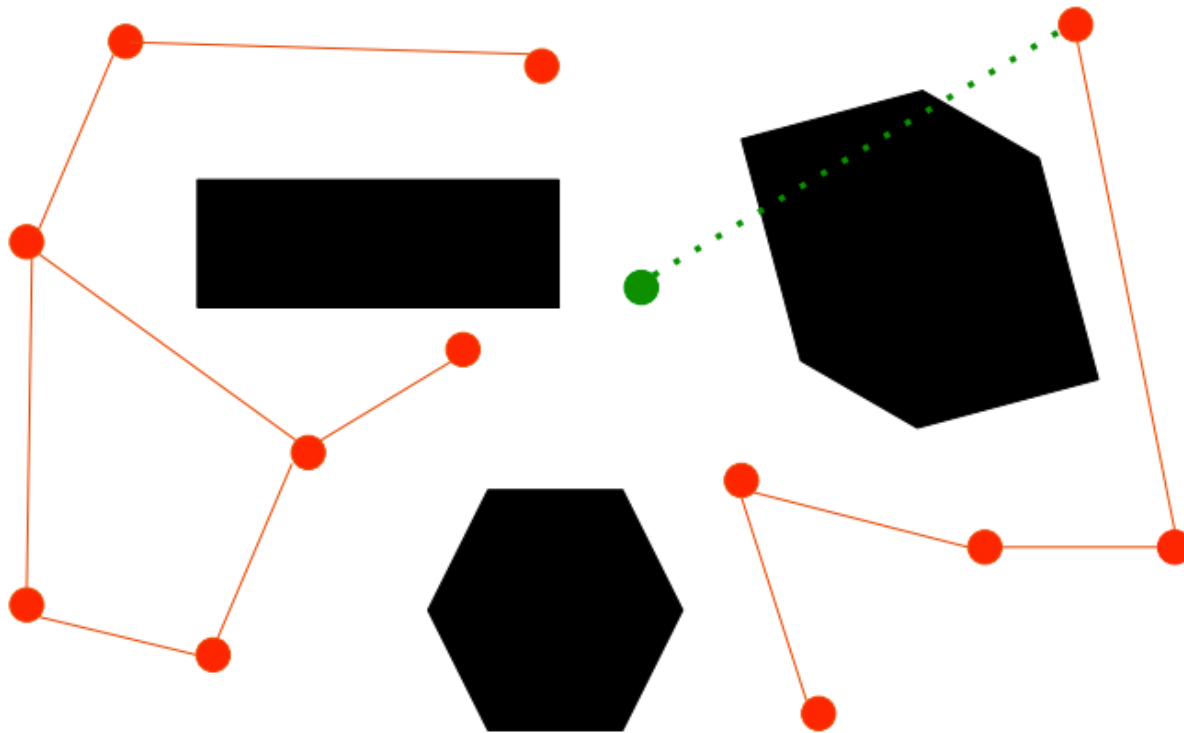
- Distance with angle

- if θ_1 and $\theta_2 \in [0, 360^\circ]$, we can use the following equation to capture the angular displacement

$$Dist(\theta_1, \theta_2) = \min(|\theta_1 - \theta_2|, (360 - |\theta_1 - \theta_2|))$$



Collision Detection

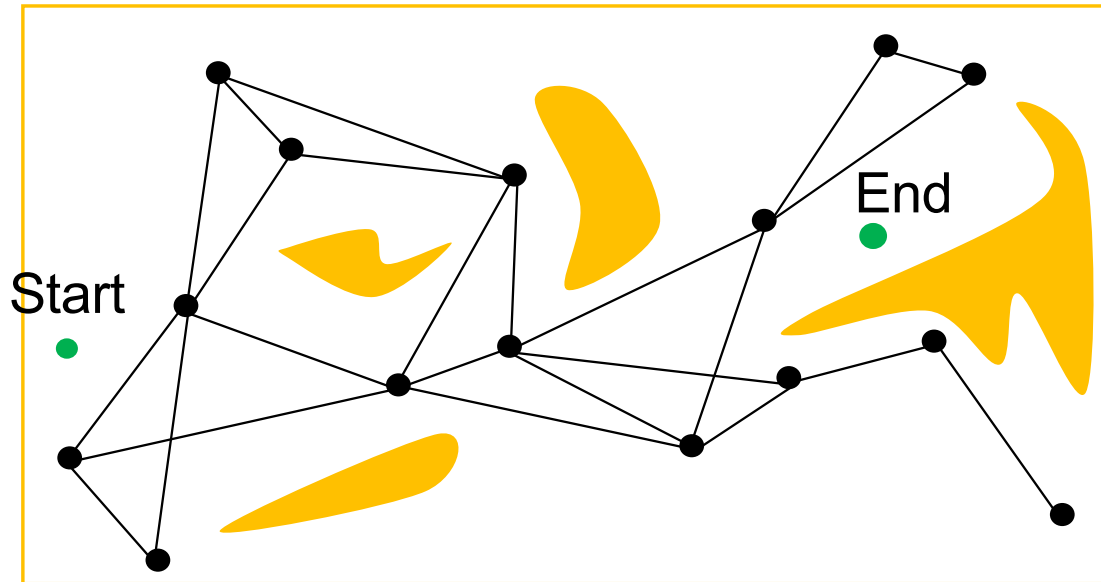


In 2D space:

- Uniformly sample points along the path
- Check if all the points are in free space
 - the path is collision free

In 3D space, please check Gilbert–Johnson–Keerthi distance algorithm

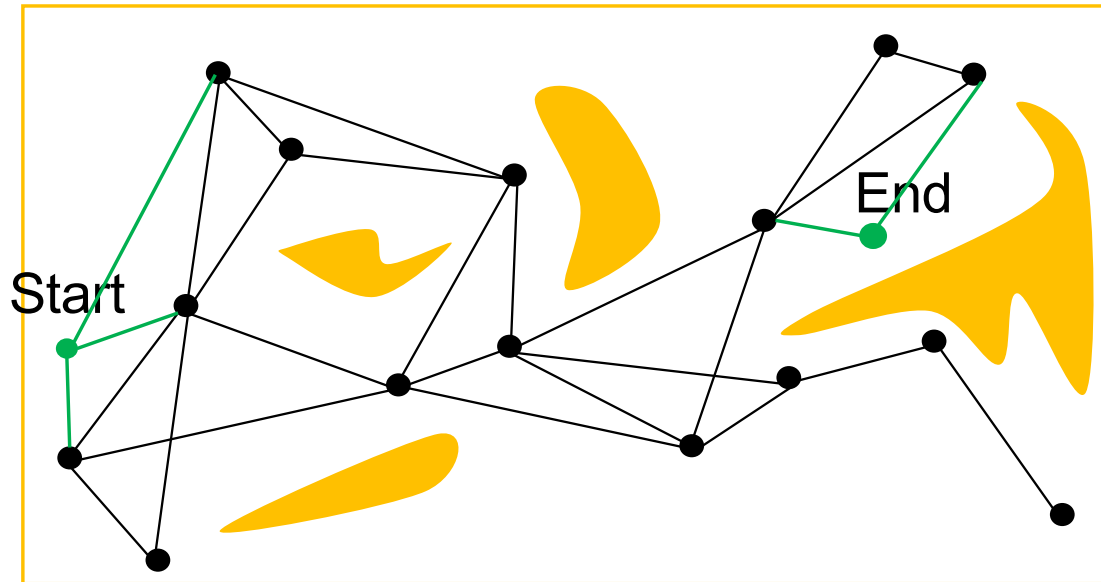
Probability Roadmap (PRM) – Phase 2



Query Phase:

Search a path given q_{init} and q_{goal} with the constructed graph

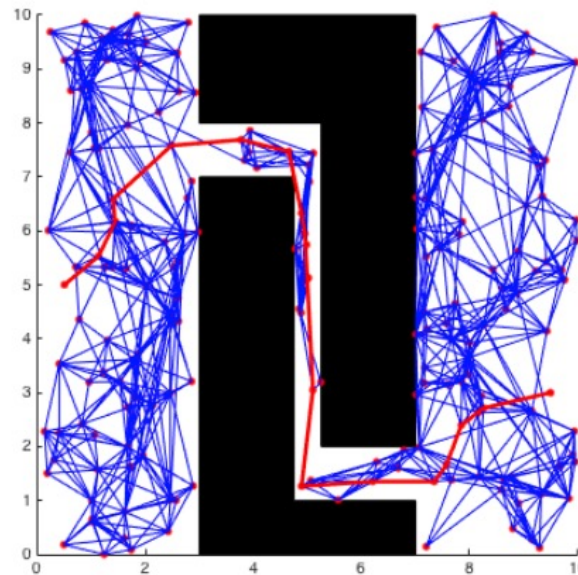
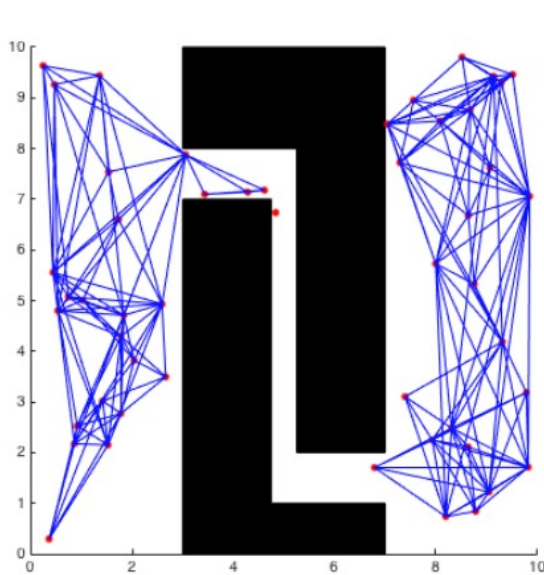
Probability Roadmap (PRM) – Phase 2



- Add these two node into the graph
- Connect possible edges to existing nodes
- Perform a graph search.

Random Sample in PRM

It is possible to have a situation where the algorithm would fail to find a path even when one exists if the sampling procedure fails to generate an appropriate set of samples.



Possible solution:

1. More Samples
2. Sample more points close to C-space obstacles

Probabilistically Complete

- It is possible to have a situation where the algorithm would fail to find a path even when the solution exists
 - Due to inappropriate samples
- If there is a route and the planner keeps adding random samples, it will eventually find a solution
 - Lots of samples...

Can we do it differently?

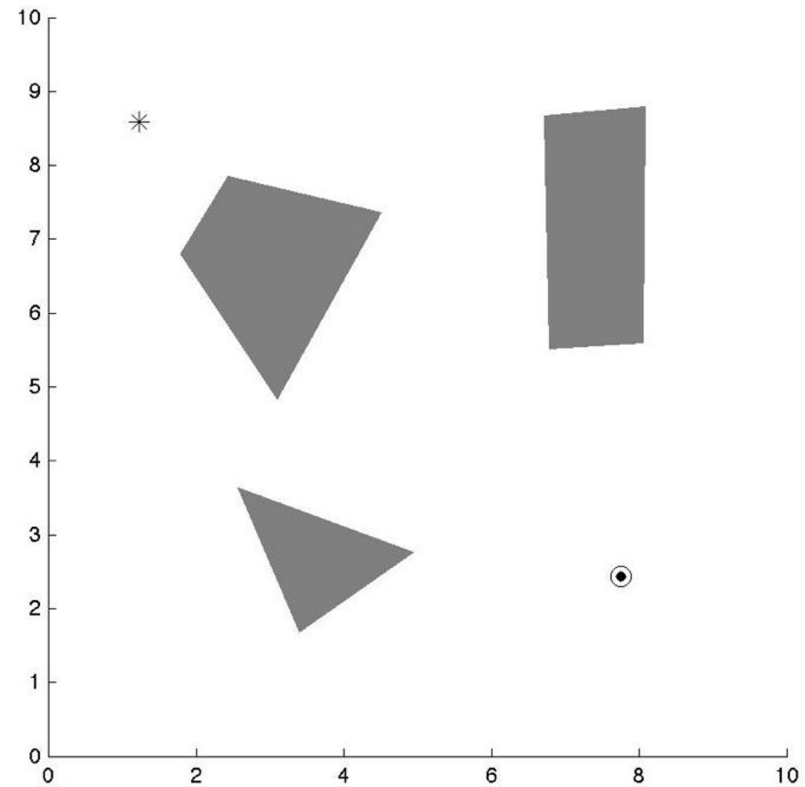
Rapidly-exploring Random Tree (RRT)

General Idea of RRT Algorithm

- Search for a path from the initial configuration to the goal configuration by **expanding a search tree iteratively**
 - Instead of a general graph
- For each step
 - samples a target configuration and expands the tree towards it
 - The sample can either be a random configuration or the goal configuration itself

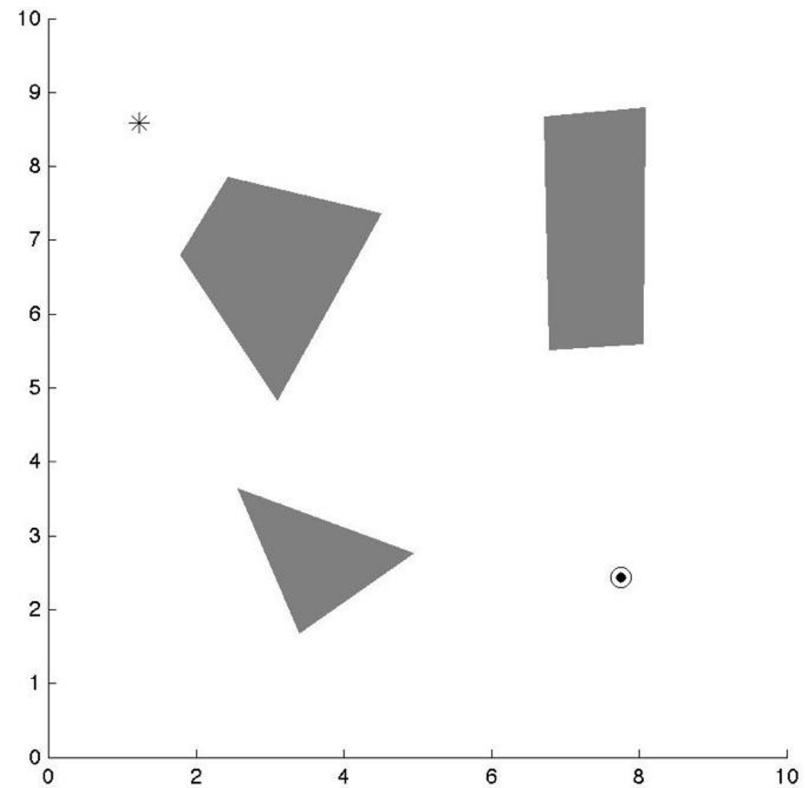
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow RandomSample()$   
   $x_{nearest} \leftarrow Nearest(G, x_{rand})$   
   $x_{new} \leftarrow Steer(x_{nearest}, x_{rand})$   
  if  $ObstacleFree(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



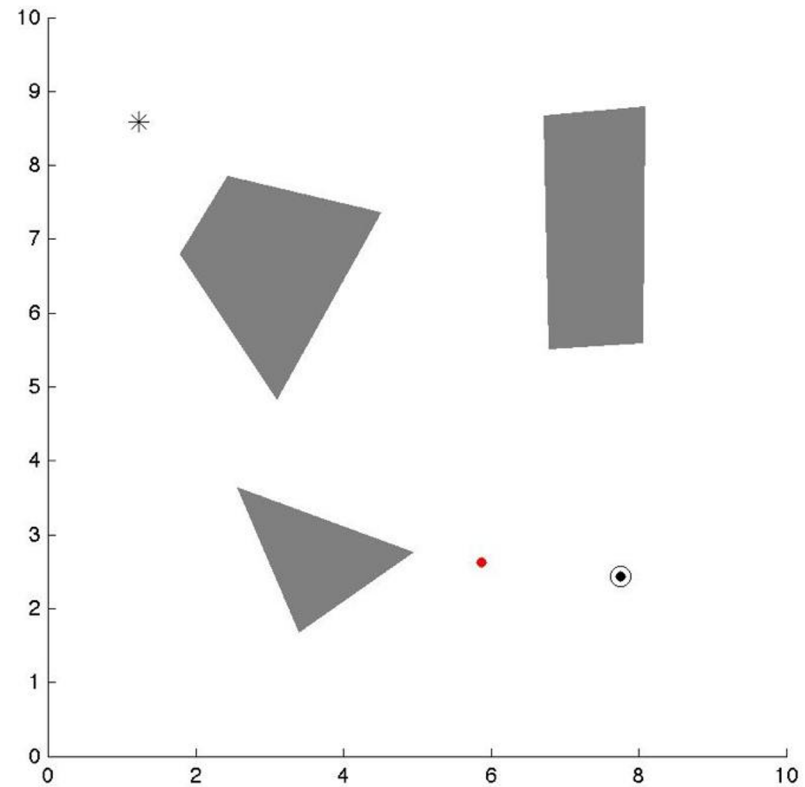
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



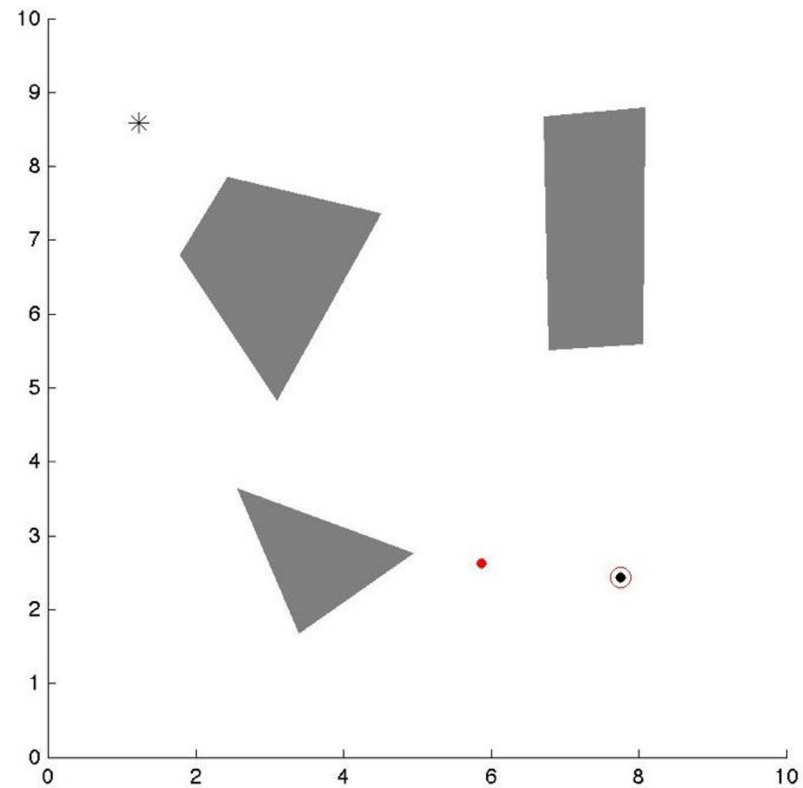
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



RRT

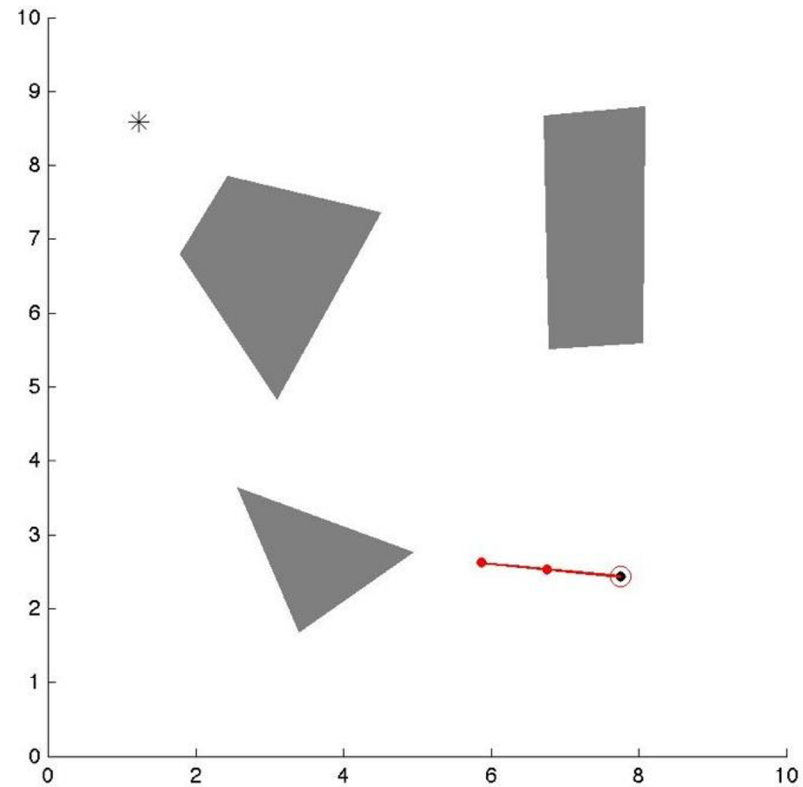
```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



RRT

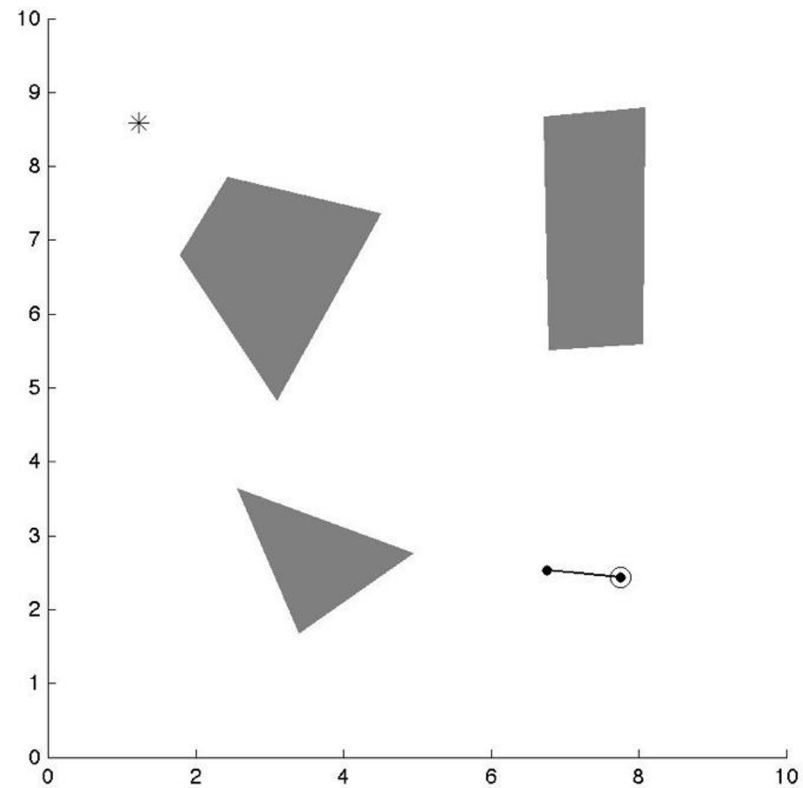
```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```

Add a stepsize
from $x_{nearest}$
to obtain x_{new}



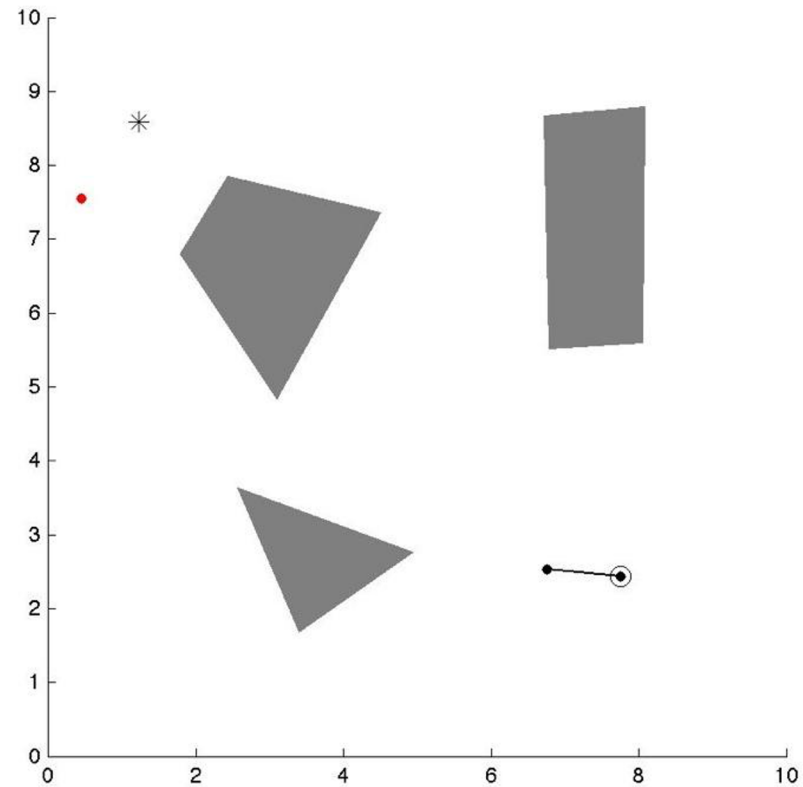
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



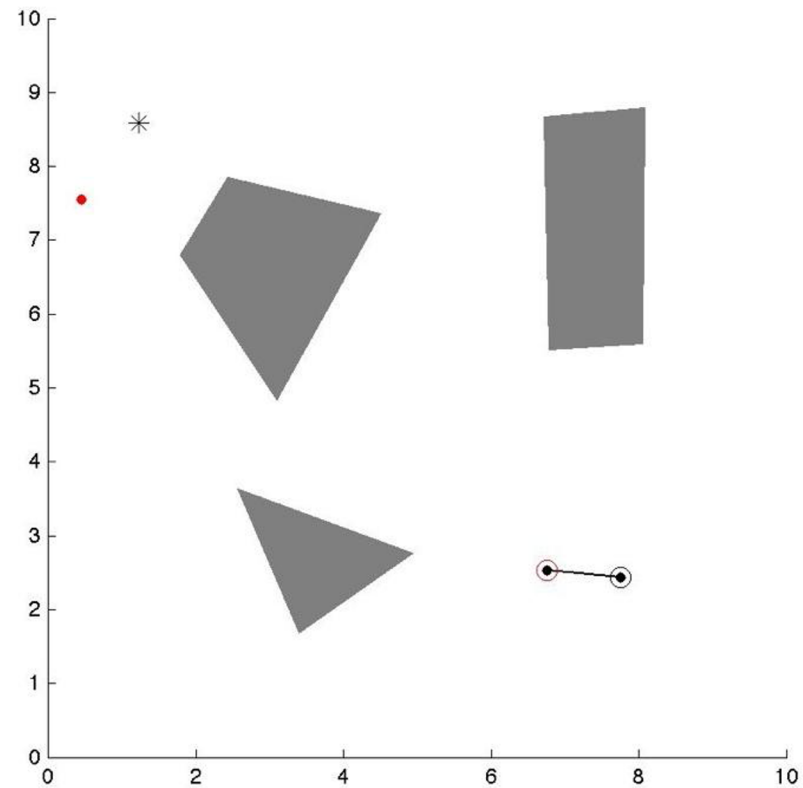
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



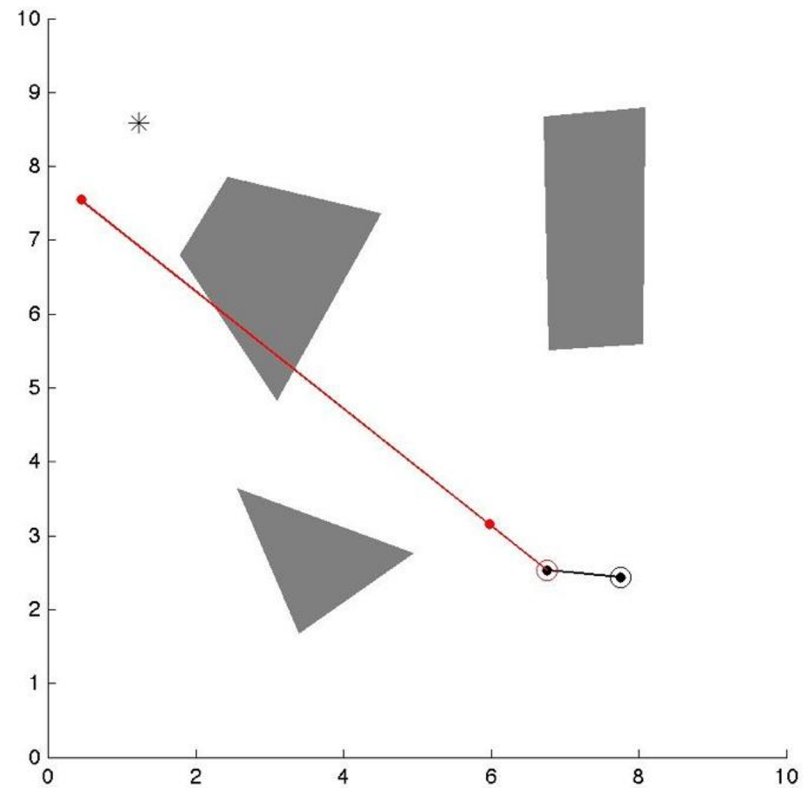
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



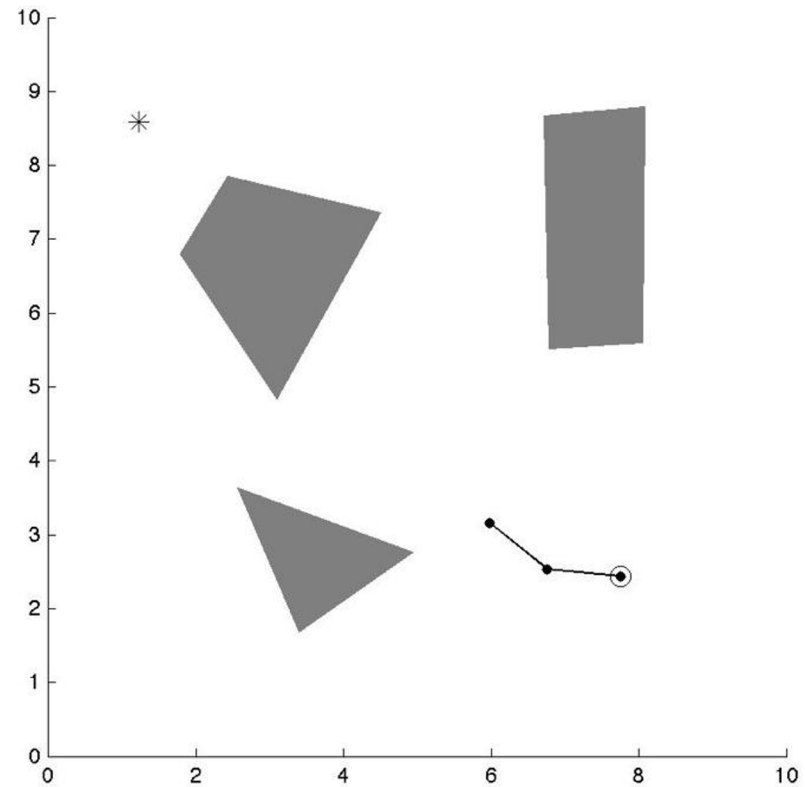
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



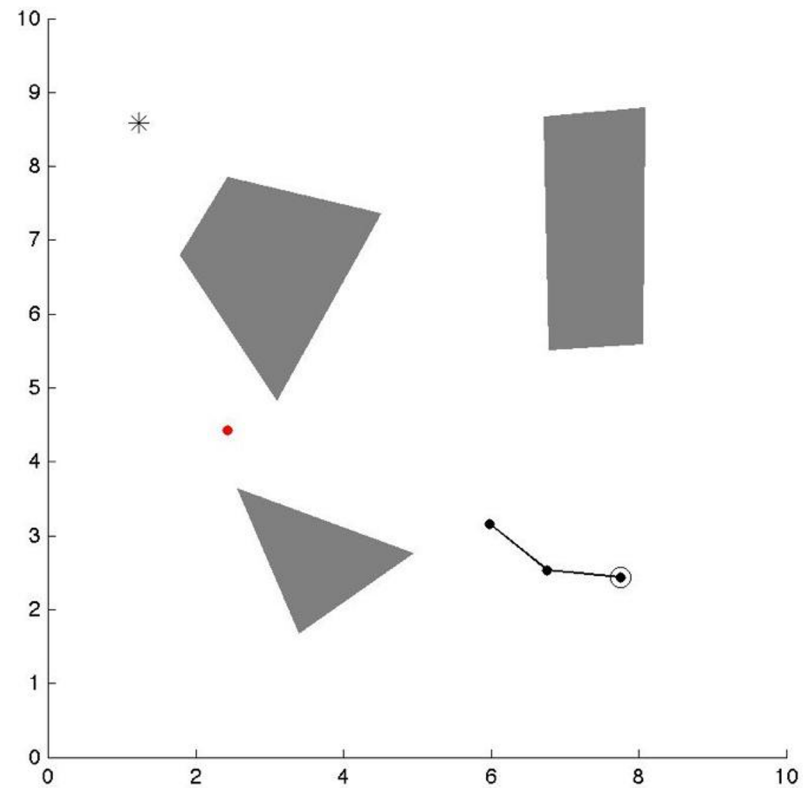
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



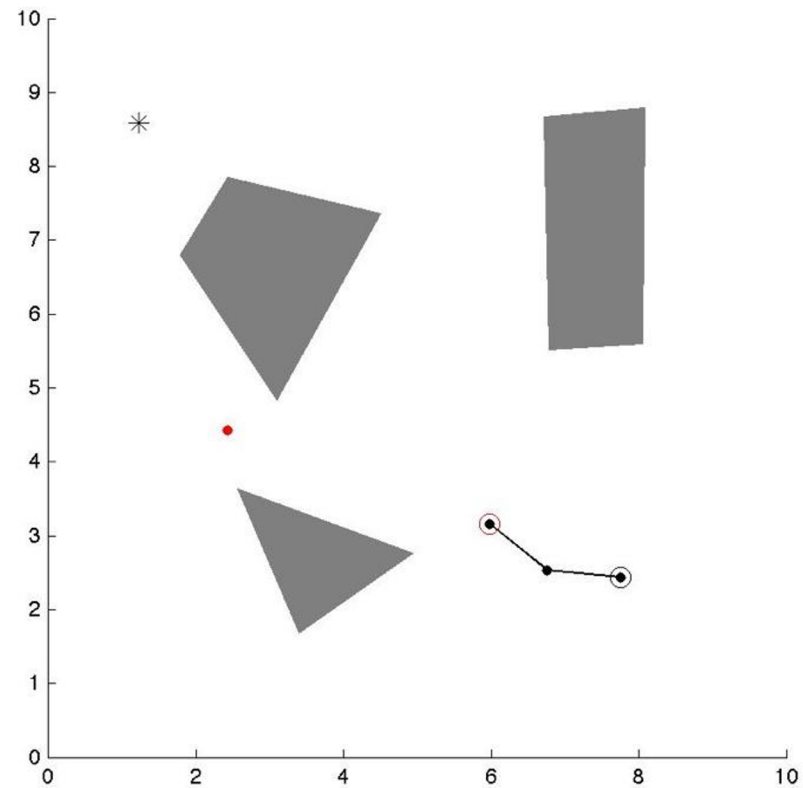
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



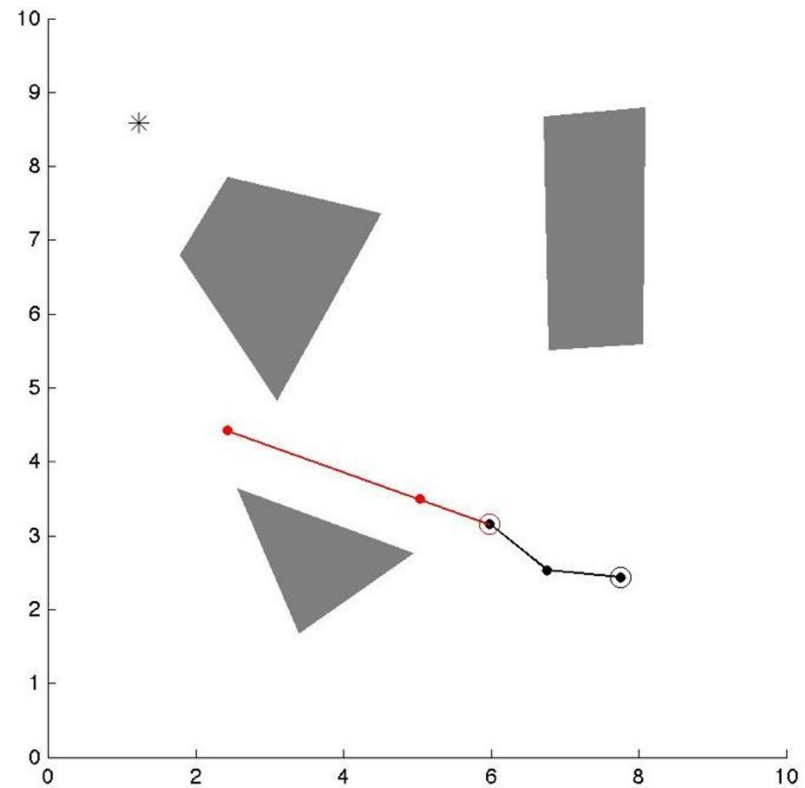
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



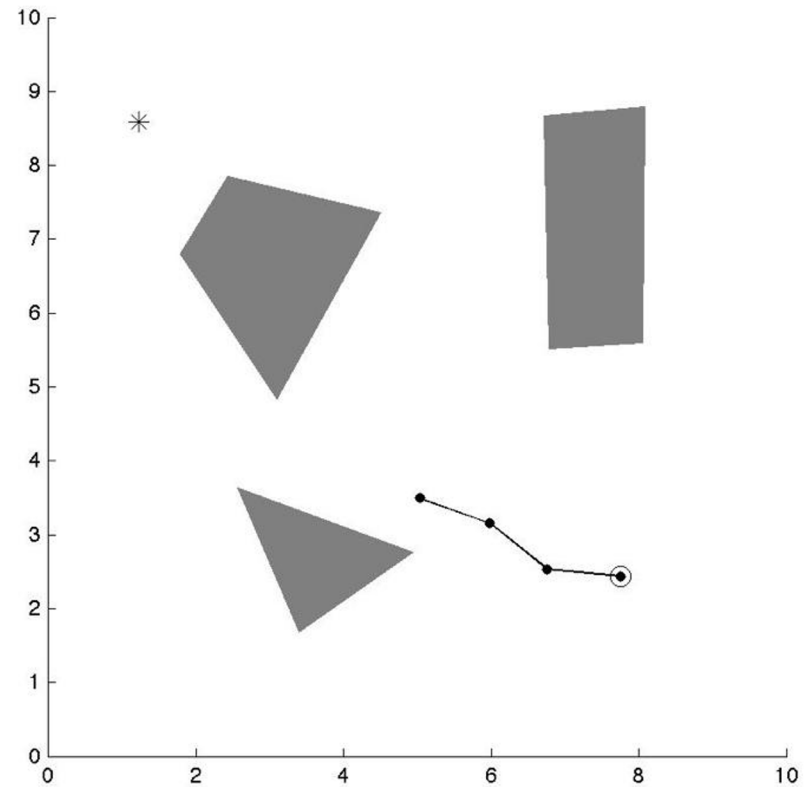
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



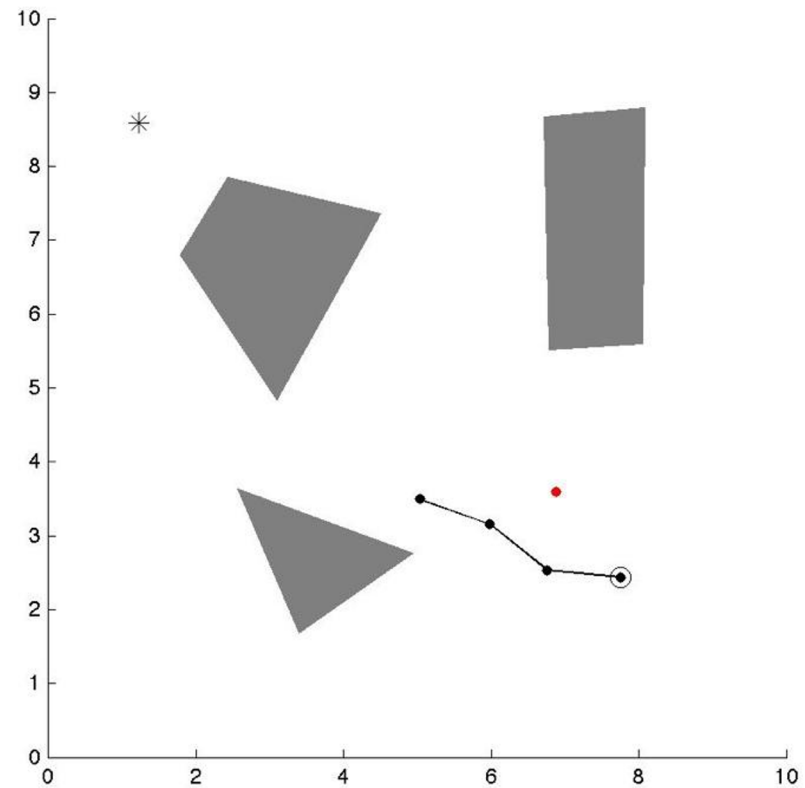
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



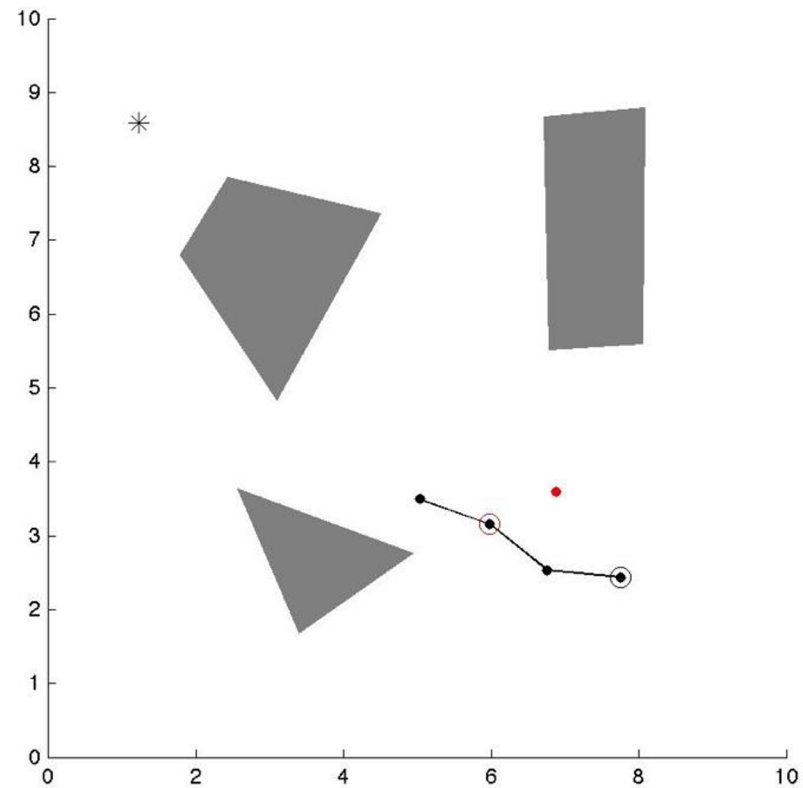
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



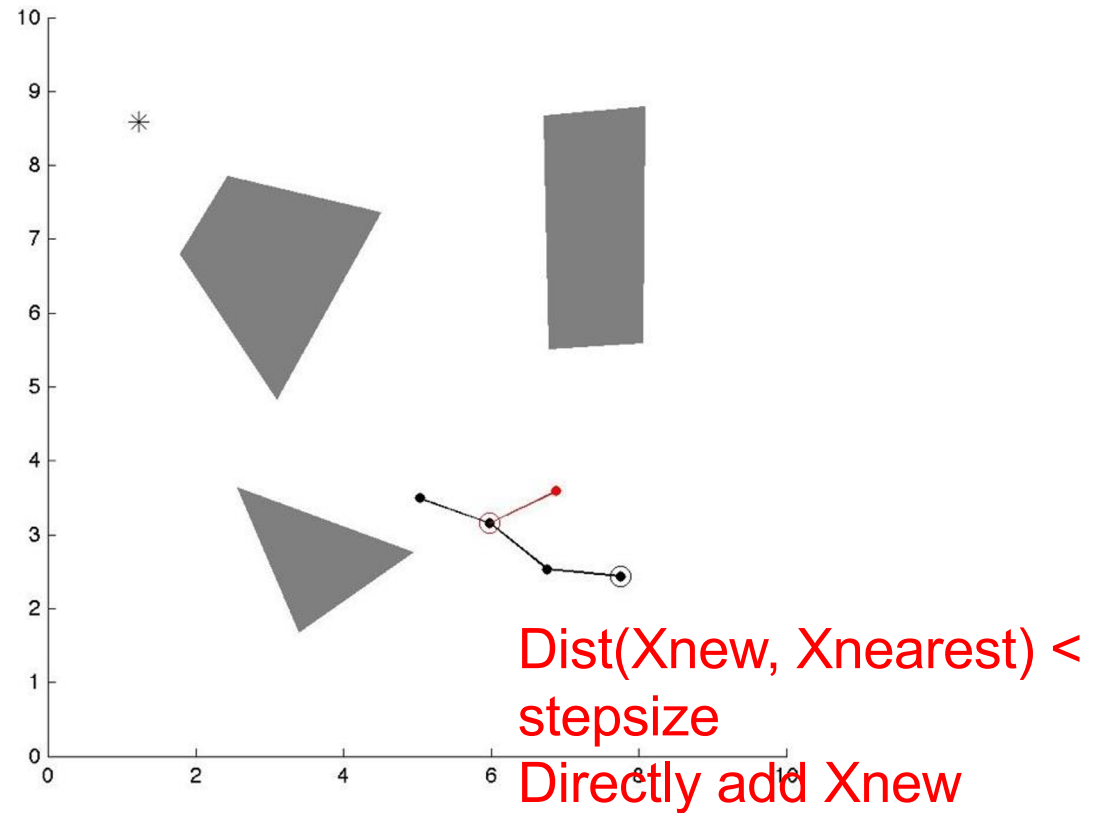
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



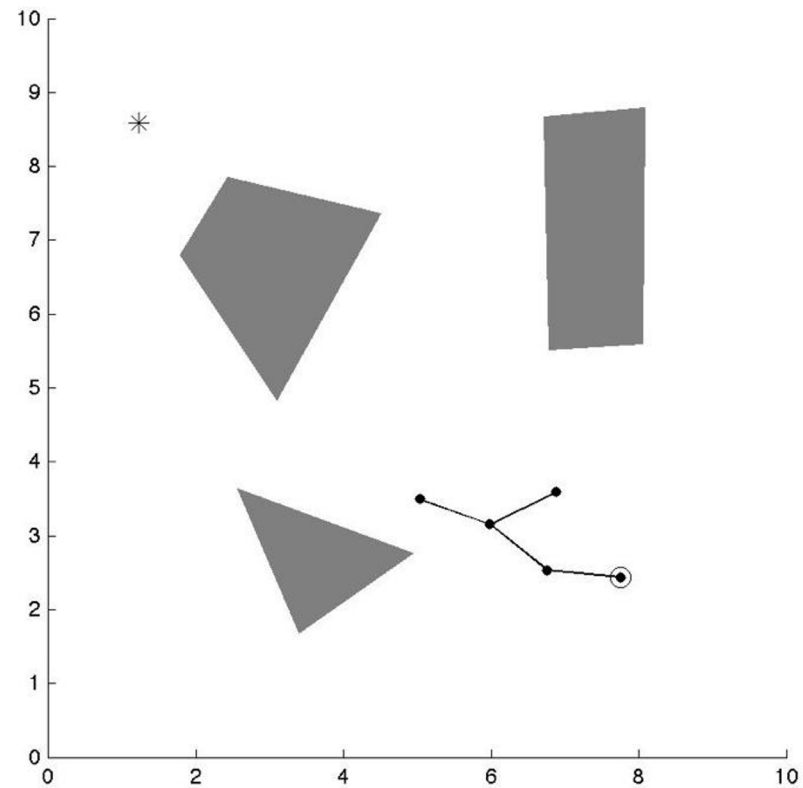
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



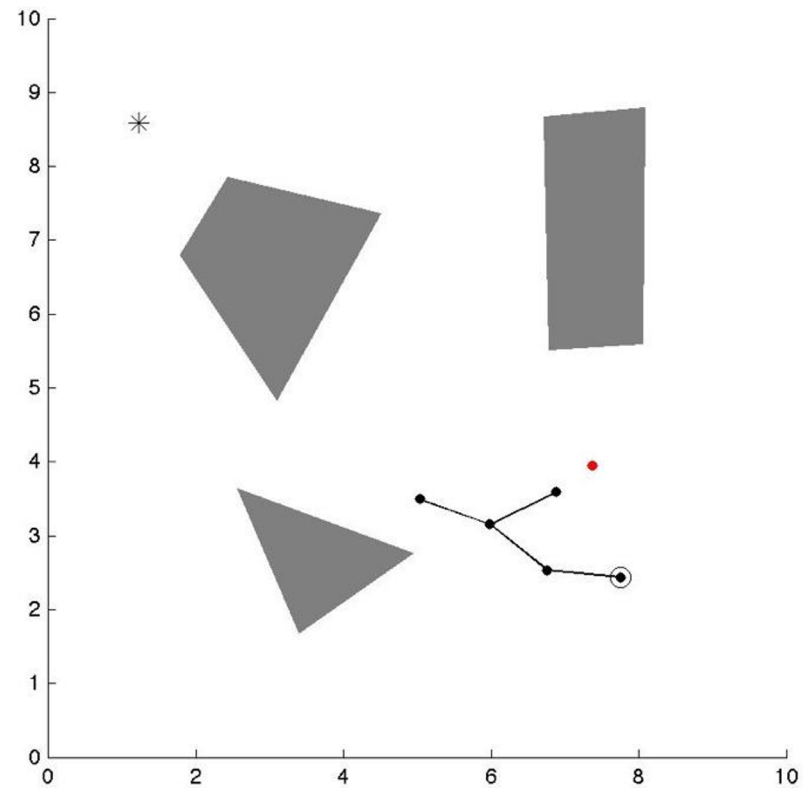
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



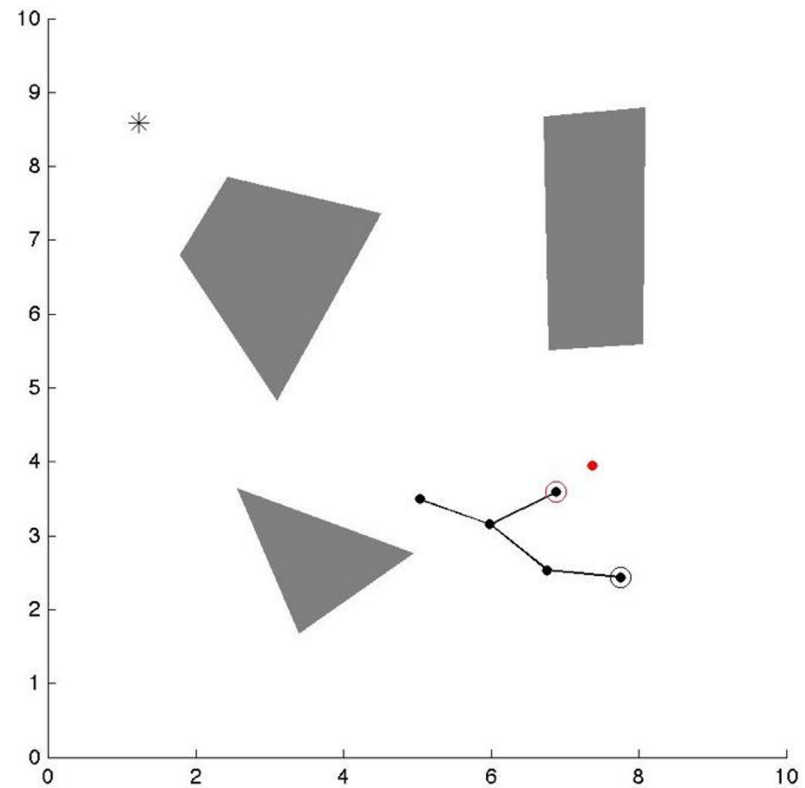
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



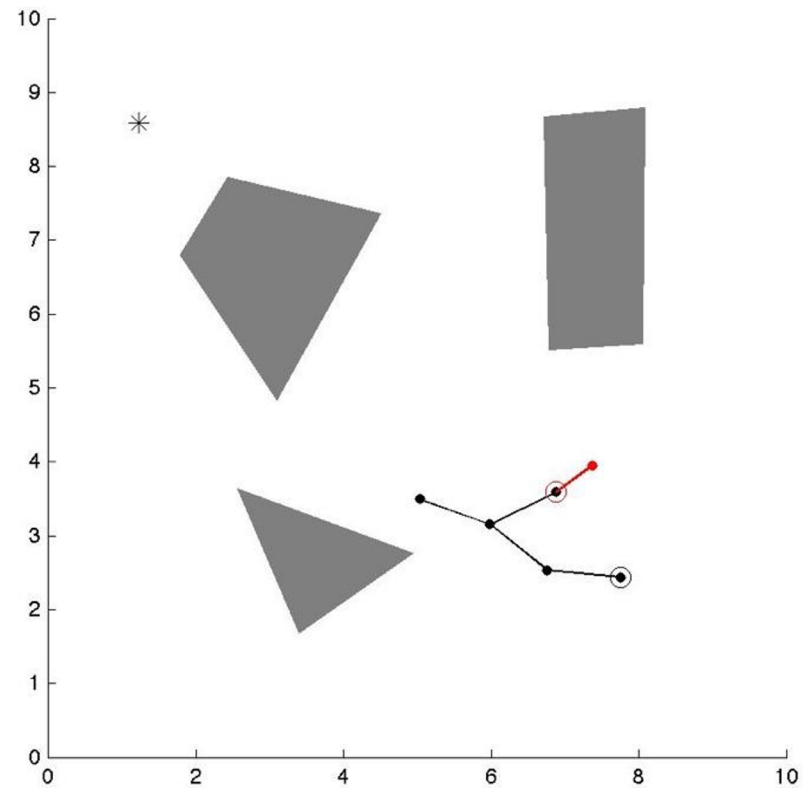
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



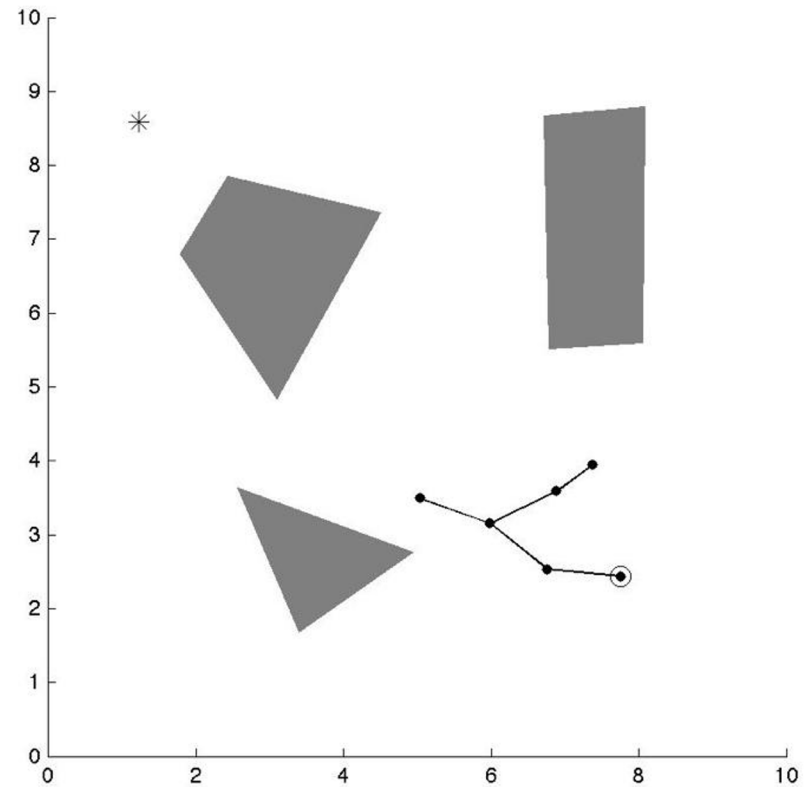
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



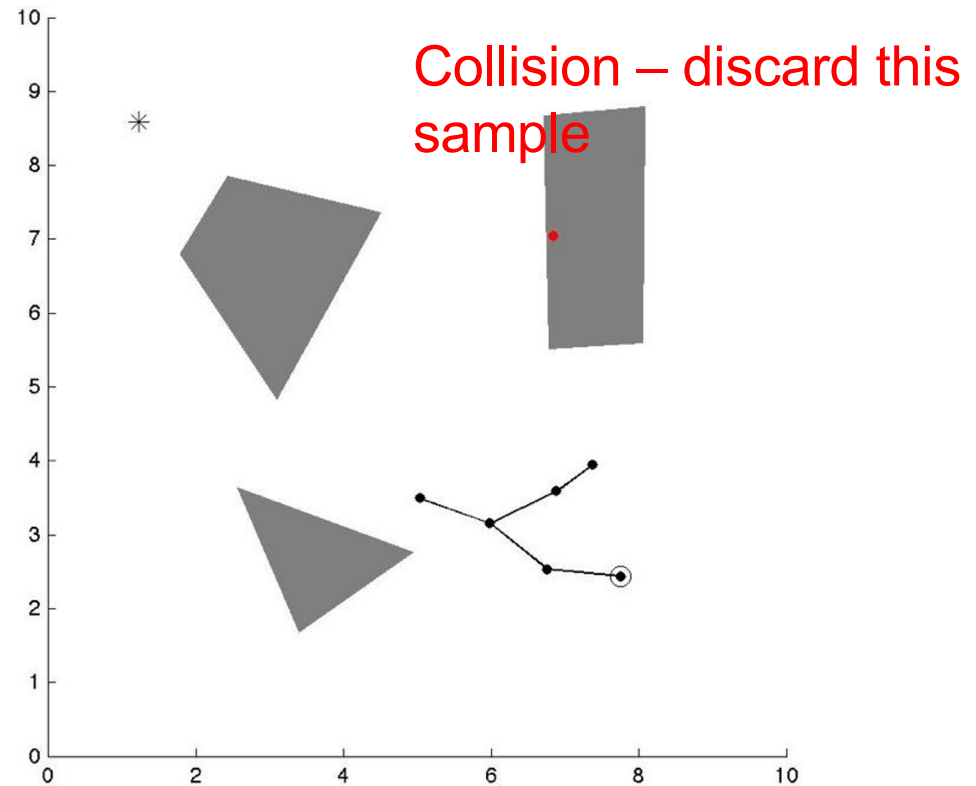
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



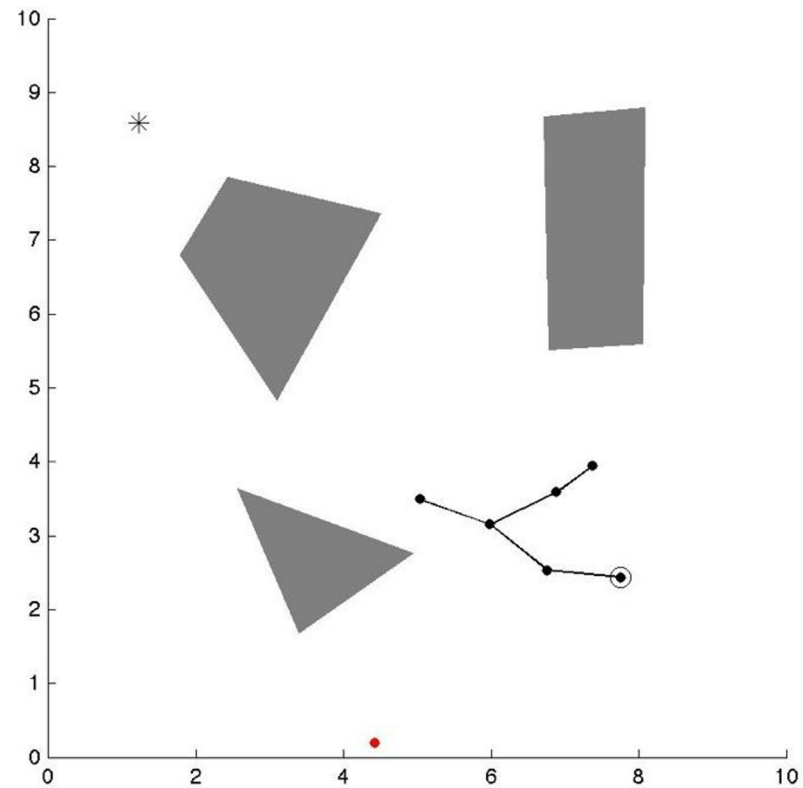
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



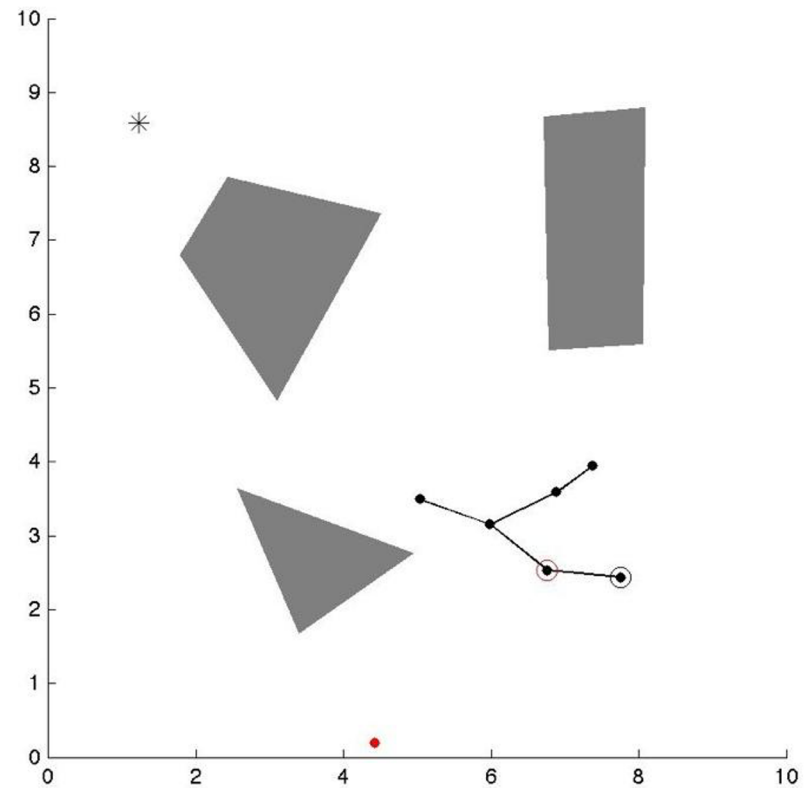
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



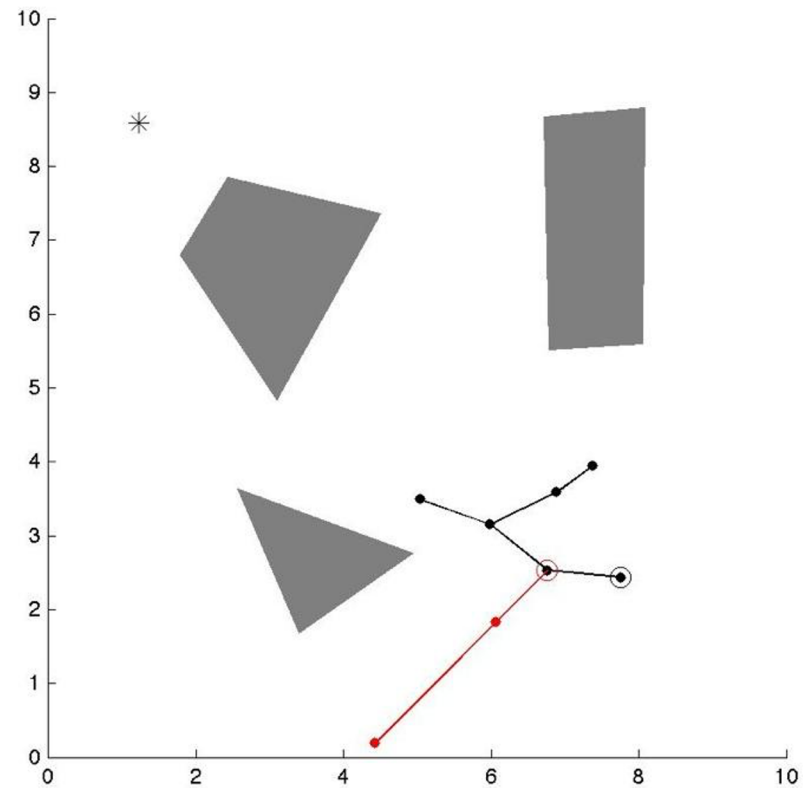
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



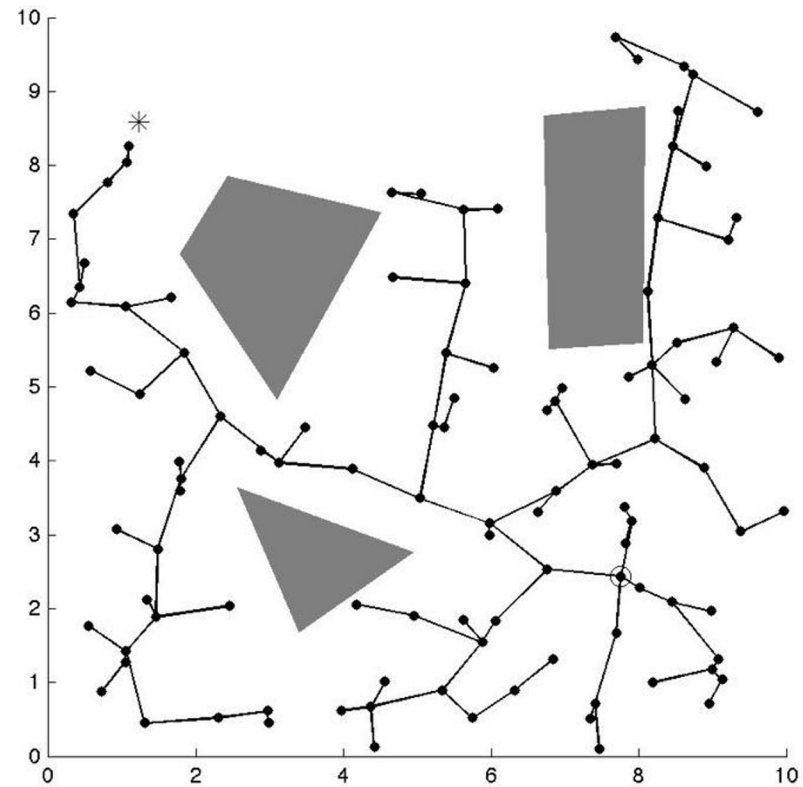
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```



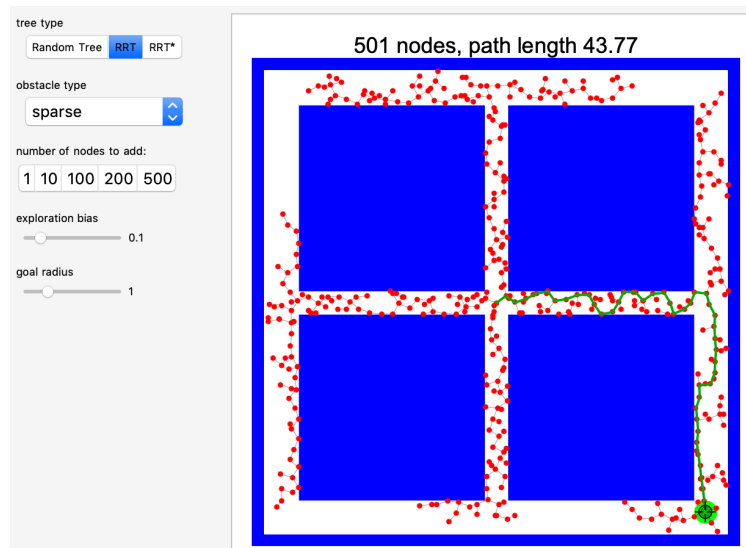
RRT

```
 $V \leftarrow \{x_{init}\}; \quad E \leftarrow \emptyset$   
for  $i = 1$  to  $N$   
   $G \leftarrow (V, E)$   
   $x_{rand} \leftarrow \text{RandomSample}()$   
   $x_{nearest} \leftarrow \text{Nearest}(G, x_{rand})$   
   $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand})$   
  if  $\text{ObstacleFree}(x_{nearest}, x_{new})$   
     $V \leftarrow V \cup \{x_{new}\}$   
     $E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$ 
```

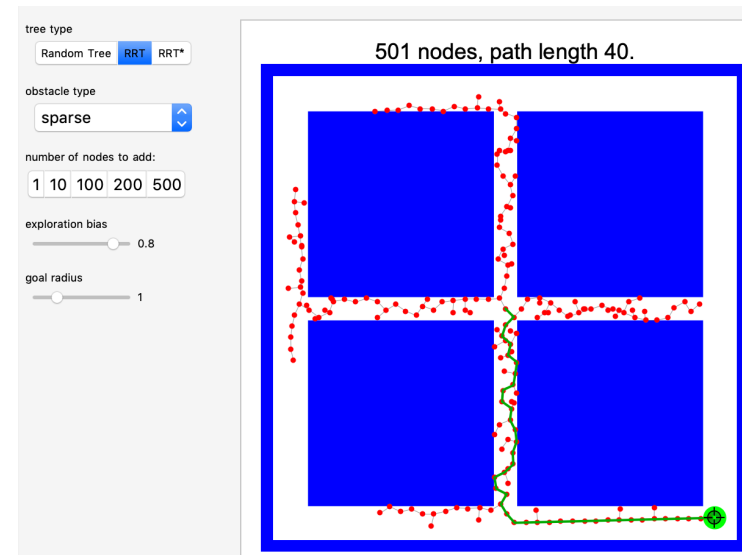


How to leverage Goal information?

- When generating a random sample, with a **probability** picking the goal instead of a random node when expanding



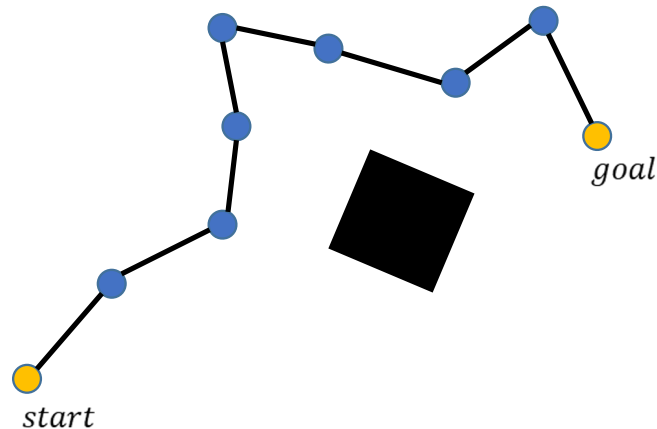
Less samples picking the goal



More samples picking the goal

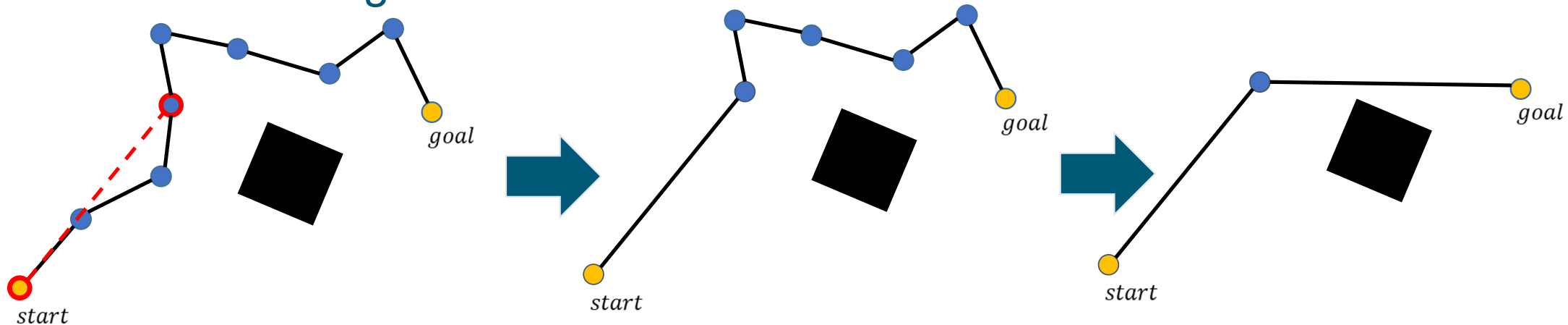
Optimizing the Path

- Paths generated by RRT are far from optimal and require additional refinement before they are usable
- We typically get the following path

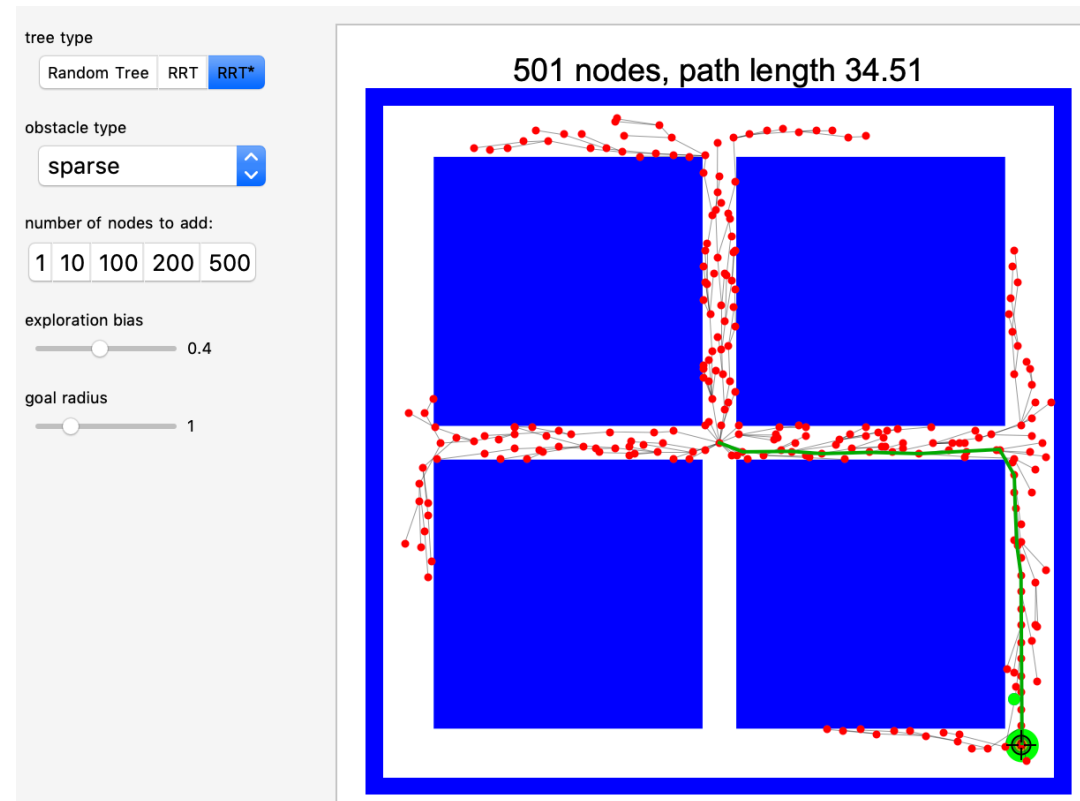
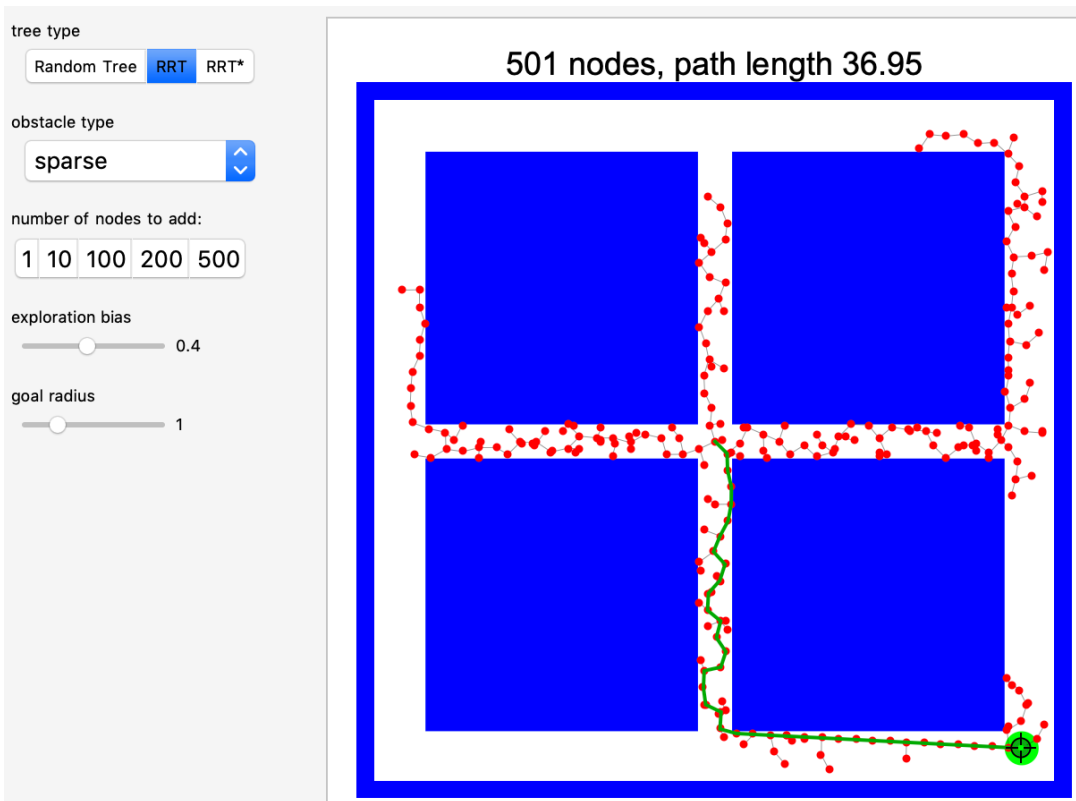


Optimizing the Path (RRT*)

- A simple way is to repeatedly pick two nodes at random and check whether they can be connected by a straight line without collision (shorten a path)
- The result is not an optimal path, but shorter and more efficient than the original

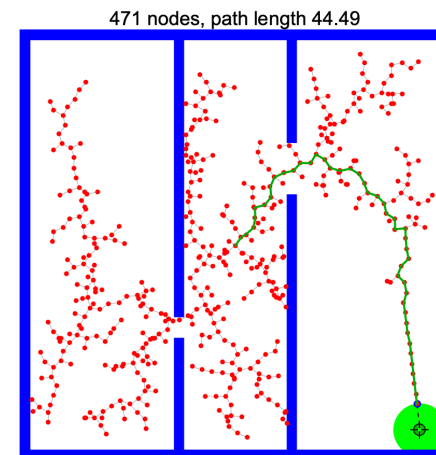
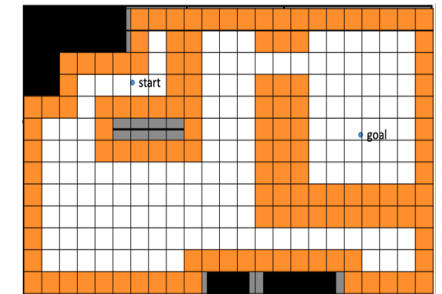
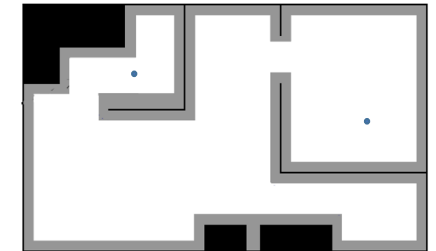
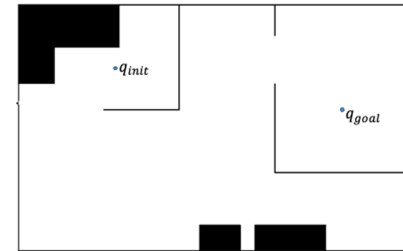


RRT vs. RRT*



Summary

- Configuration space with obstacles
 - Robot as a point
- Turn configuration space into a graph
 - Occupancy grid
- Path planning as search
 - Uninformed or informed search (A*)
- Sample based path planning
 - PRM, RRT, RRT*



START

9	10	9	8		
8	9	8	7	6	
7			6	5	4
6	5				3
5	4		2	1	2
	3	2	1	0	1
		3	2	1	2





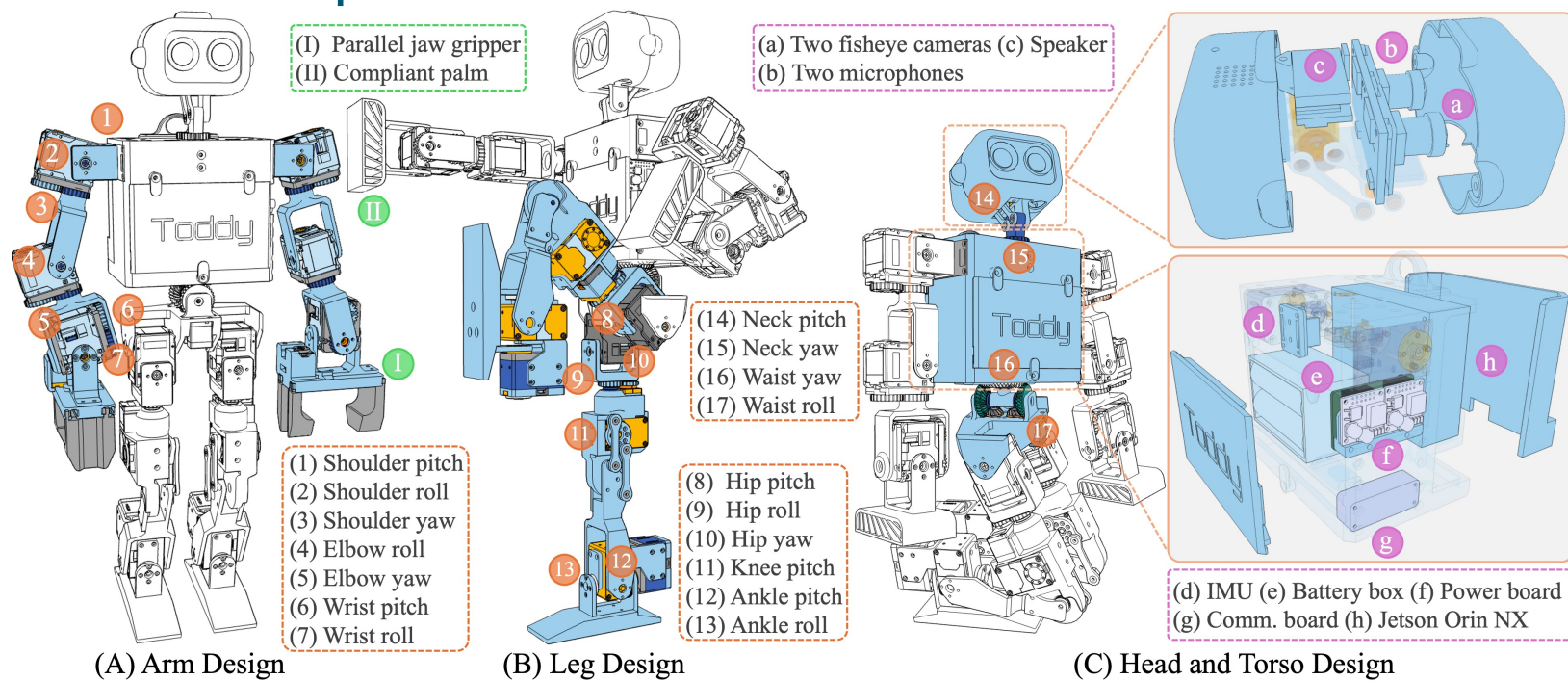
*Dynamics: the relationship between the forces acting on a robot and the resulting motion of the robot

What is the underlying control scheme behind the system?

How can we model its dynamics*?

System Identification

- ToddlerBot: Open-Source ML-Compatible Humanoid Platform for Loco-Manipulation



Laser Scanner Jitter

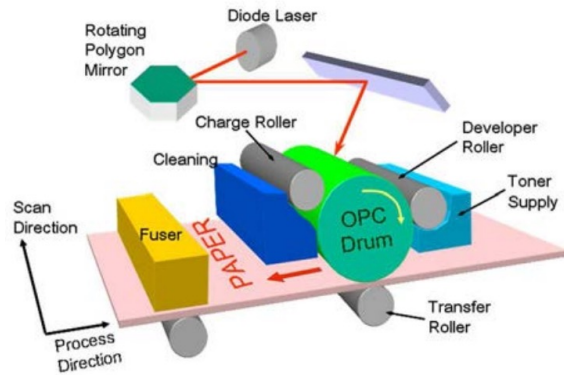


Fig. 1.1. Architecture of a typical electrophotographic printer. [1]

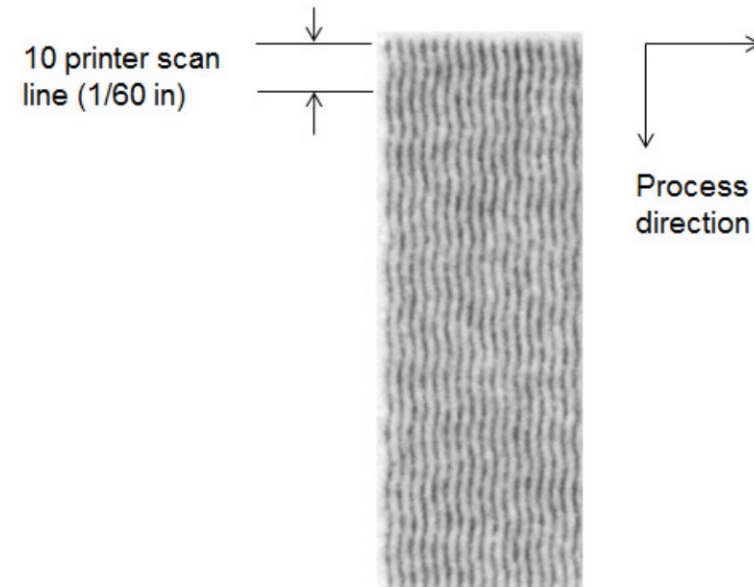


Figure 1. *Straight lines printed in the process direction appear to be wiggly instead of straight. The length of each straight line is 100 printer-addressable pixels, and the periodic pattern repeats every 10 pixels because the polygon mirror in our target laser printer has 10 facets. The printer resolution is 600 dpi.*

Mathematical Modeling

Mathematical Model of Laser Scanner Jitter

We first develop the general mathematical model of periodic laser scanner jitter in a two-dimensional continuous-space for a continuous-tone image, and extend it to the special case of a sinusoidal displacement in the next section. We denote the continuous-tone image by $f(x, y)$. Here x corresponds to the process (vertical) direction, and y corresponds to the scan (horizontal) direction. We define a new image $g(x, y)$, with periodic laser scanner jitter, according to

$$g(x, y) = f(x, y - d(x)), \quad (1)$$

where the displacement $d(x)$ is periodic in x . Let $F(u, v)$ denote the two-dimensional Continuous-Space Fourier Transform

Periodic Laser Scanner Jitter with Sinusoidal Displacement

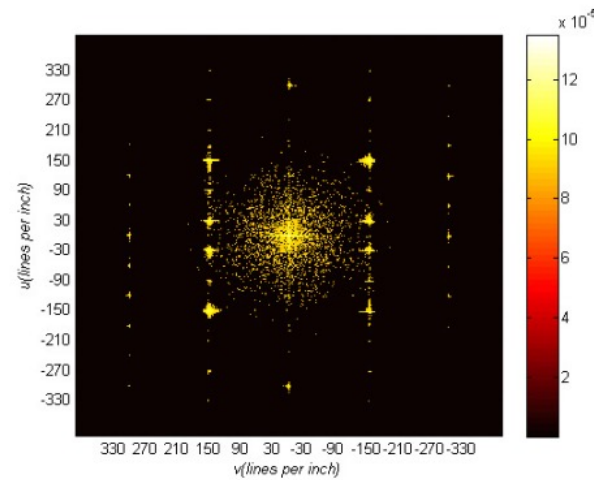
In this section, we extend the model to the special case where the periodic displacement is a sinusoid function $d(x) = A \sin(2\pi x/X)$. Here A is a constant scale factor that is the peak jitter displacement, and X is the period of the laser scanner jitter. With an approach similar to that used in the previous section, we

<https://drive.google.com/file/d/0B9mTQnwIYIDzVDVQUFM4cjZKbzQ/view?resourcekey=0-FmLYIR-YM-nm5CwO0fy3wQ>

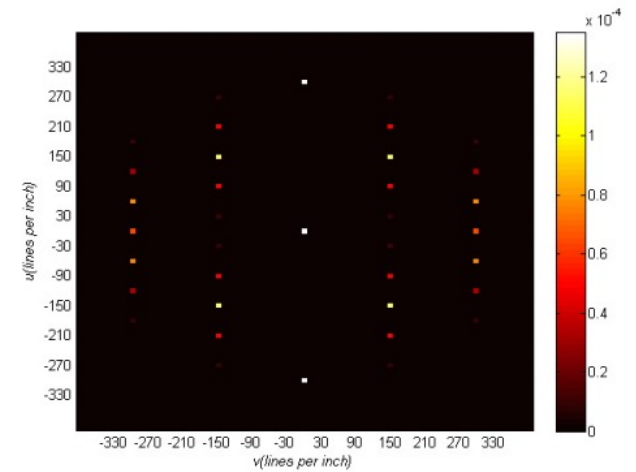
Quantitative Experiments



(a)



(a)



(b)

<https://drive.google.com/file/d/0B9mTQnwIYIDzVDVQUFM4cjZKbzQ/view?resourcekey=0-FmLYIR-YM-nm5CwO0fy3wQ>